Placement Reading Template - Compact

Placement / OA Coding

October 2026

What this file is for

This is a **memory-refresh companion to `blind_typing_codes_upgraded.md`**, not a replacement DSA book.

Your default should be: - **Blind typing:** build muscle memory for a small P0/P1 set. - **Reading:** inspect the same canonical implementations for 10-30 minutes, especially variants and edge cases. - **Do not read 40+ pages linearly during meals.** Read one section at a time and rotate.

The existing Blind Typing template already separates high-priority placement drills from heavier techniques and says the Reading/Memory Vault is for passive inspection rather than daily blind typing. This compact file makes that split practical.

Recommended 1-hour routine

Block	Time	What to do
Core warmup	2-5 min	Math/bits from Blind Typing
Blind typing P0	25-30 min	3-5 high-frequency cards
Blind typing P1	10-15 min	1 harder/variant card
Meal reading	10-20 min	Read the corresponding section below
Final recall	2 min	Close the file and state the invariant/edge case aloud

Important: do not try to finish this document in one sitting. A 10-20 minute meal-read should cover roughly one cluster.

Placement priority map

Tier	Blind typing	Reading
P0 - must type	Binary bounds, Two Sum, 3Sum, Kadane, Next Permutation, rotated binary search, merge intervals, sliding max, NGE, min-window, KMP, BFS/DFS, Dijkstra, Kahn, DSU, LCA, 0/1 Knapsack, subset sum, LIS, LCS, edit distance, House Robber, Fenwick, generic Segment Tree	Same code; focus on invariant + one variant
P1 - type weekly	Trapping Rain Water, Allocate Pages, 0-1 BFS, Kruskal, Trie, 3Sum, Unique Paths, matrix exponentiation, generic lazy Segment Tree, max-subarray Segment Tree	Read the variants and failure conditions
P2 - read only / occasional typing	Bellman-Ford, Floyd-Warshall, min+count Segment Tree, max-subarray Segment Tree, advanced lazy variants	Read when relevant to OA/company style

Tier	Blind typing	Reading
P3 - break-glass	Segment Tree Beats, Persistent Segment Tree, Merge Sort Tree, HLD, Li Chao, digit DP, suffix array/SAM/Aho-Corasick, Gaussian/SOS/Mo's	Do not spend normal meal time here

The key variant rule

When you recognize a problem, **do not search for a new algorithm first**. Ask: "Which known template changes by one operation/state?"

Common changes are summarized before the code cards.

Binary search variants

- Exact target -> ordinary binary search.
- First `>= x` -> `lower_bound`.
- First `> x` -> `upper_bound`.
- Maximize/minimize an answer -> write a monotone `ok(mid)` predicate and binary-search the answer.
- Rotated sorted array -> identify which half is sorted.
- Aggressive Cows -> maximize a feasible distance; change only the predicate.
- Allocate Pages -> minimize a feasible maximum load; change only the predicate.

Sliding-window variants

- Longest distinct substring -> maintain last seen index / frequency.
- Minimum window -> maintain required counts and a satisfied counter.
- Fixed-size maximum -> monotonic deque.
- Fixed-size sum/average -> prefix sum or running sum.

Graph shortest-path choice

- Unweighted -> BFS.
- Edge weights only `0/1` -> 0-1 BFS.
- Non-negative arbitrary weights -> Dijkstra.
- Negative edges / negative-cycle detection -> Bellman-Ford.
- All pairs, small `V` -> Floyd-Warshall.

DP loop-direction rule

- 0/1 knapsack -> capacity goes **down**.
- Unbounded knapsack -> capacity goes **up**.
- Subset boolean/counting -> usually capacity goes down when each item is used once.
- LIS strict increasing -> `lower_bound`.
- LIS non-decreasing -> `upper_bound`.
- LCS -> subsequence; LCS-substring -> contiguous match resets on mismatch.

Fenwick variants

The same BIT structure covers: - point add + prefix/range sum, - frequency counting after coordinate compression, - inversion count, - count smaller after self, - Create Sorted Array through Instructions, - prefix maximum/minimum only under restricted monotone-update conditions, - `k`-th frequency by Fenwick binary lifting when the stored values are non-negative counts.

Segment Tree operation rule

For ordinary range-query segment trees, the merge must be **associative**. Commutativity is optional.

Aggregate	Identity	Associative	Commutative	Inverse useful?	Typical template
Sum	`0`	Yes	Yes	Yes	Fenwick / SegTree
XOR	`0`	Yes	Yes	Yes	Fenwick / SegTree
GCD	`0`	Yes	Yes	No	SegTree / Sparse Table
Min	`+INF`	Yes	Yes	No	SegTree / Sparse Table
Max	`-INF`	Yes	Yes	No	SegTree / Sparse Table
String concat	`""`	Yes	No	No	Keep left/right order
Matrix multiply	`I`	Yes	No	Usually no	Keep left/right order
Subtraction	N/A	**No**	No	No	Not a monoid merge
Division	N/A	**No**	No	No	Not a monoid merge

For lazy propagation, the update also needs to be a valid action on the stored node summary, and lazy tags must compose in the correct chronological order. Range `chmin/chmax` is the classic case where a plain single lazy value is insufficient; that is why Segment Tree Beats exists.

Code cards

Code convention: Reading cards may repeat a useful Blind Typing implementation. Generic code stays ID-free; problem-based cards use only a short description, the supplied link, and the actual code when a code card is included.

Core: modular arithmetic

```cpp
// Core placement utilities: binary exponentiation, gcd, extended gcd, and modular inverse.
long long qexp(long long a, long long b, long long m = MOD) {
    long long res = 1; a %= m;
    while (b) { if (b & 1) res = (long long)((__int128_t)res * a % m); a = (long long)((__int128_t)a * a
     ↪  % m); b >>= 1; }
    return res;
}
long long gcd(long long a, long long b) { return b ? gcd(b, a % b) : a; }
long long ext_gcd(long long a, long long b, long long &x, long long &y) {
    if (!b) { x = 1; y = 0; return a; }
    long long x1, y1, d = ext_gcd(b, a % b, x1, y1);
    x = y1; y = x1 - y1 * (a / b); return d;
}
long long inv_mod(long long a, long long m) {
    long long x, y; long long g = ext_gcd(a, m, x, y);
    if (g != 1) return -1;
    return (m + x % m) % m;
}
```

Core: bit fundamentals

```cpp
// These are the bit operations worth having in muscle memory; built-ins handle popcount/leading/trailing
 ↪  zero counts.
#define getbit(x,i) (((x)>>(i))&1LL)
#define setbit(x,i) ((x)|(1LL<<(i)))
#define clearbit(x,i) ((x)&(~(1LL<<(i))))
#define togglebit(x,i) ((x)^(1LL<<(i)))
#define lowbit(x) ((x)&(-(x)))
```

```
#define strip_lowbit(x) ((x)-((x)&(-(x))))
#define is_pow2(x) ((x)&&(((x)&((x)-1LL))==0))
#define is_allones(x) ((x)&&(((x)&((x)+1LL))==0))
#define all_ones(n) ((1LL<<(n))-1LL)
```

Range query: Fenwick Tree

```
// Use 1-based indexing; the same tree handles sums and, after coordinate compression, frequency/count
 ↪  problems.
struct BIT {
    int n; vector<long long> ft;
    BIT(int n) : n(n), ft(n + 1, 0) {}
    void upd(int x, long long v) { for (; x <= n; x += lowbit(x)) ft[x] += v; }
    long long sum(int x) { long long res = 0; for (; x; x -= lowbit(x)) res += ft[x]; return res; }
    long long qry(int l, int r) { return sum(r) - sum(l - 1); }
};
```

Fenwick: range add + range sum (two BITs)

Use this when **updates add x to every a[i] in [l,r)**, while queries ask the sum of any `[l,r)`. The initial-vector constructor preserves the starting array. This is often simpler than lazy propagation for this exact operation pair.

```
struct RangeAddRangeSumBIT {
    int n; vector<long long> b1, b2;
    explicit RangeAddRangeSumBIT(int n) : n(n), b1(n + 1), b2(n + 1) {}
    explicit RangeAddRangeSumBIT(const vector<long long>& a)
        : RangeAddRangeSumBIT((int)a.size()) {
        for (int i = 0; i < n; ++i) addRange(i, i + 1, a[i]);
    }
    void add(vector<long long>& b, int i, long long x) {
        for (; i <= n; i += i & -i) b[i] += x;
    }
    long long sum(const vector<long long>& b, int i) const {
        long long ans = 0;
        for (; i > 0; i -= i & -i) ans += b[i];
        return ans;
    }
    void addRange(int l, int r, long long x) { // add x to [l,r)
        if (l >= r) return;
        add(b1, l + 1, x); add(b2, l + 1, x * l);
        if (r < n) { add(b1, r + 1, -x); add(b2, r + 1, -x * r); }
    }
    long long prefixSum(int r) const { // sum of a[0..r), r in [0,n]
        return 1LL * r * sum(b1, r) - sum(b2, r);
    }
    long long sumRange(int l, int r) const { // sum of [l,r)
        return prefixSum(r) - prefixSum(l);
    }
};
```

Call: `RangeAddRangeSumBIT fw(a); fw.addRange(1,4,7); long long s=fw.sumRange(0,4);`

Why two BITs: `b1` stores difference events; `b2` stores each event weighted by its zero-based position. Prefix sum is `r * sum(b1,r) - sum(b2,r)`. All public ranges are zero-based and half-open.

Variants: replace the stored quantity with frequencies for inversion/count-smaller problems; use coordinate compression first. For k-th by frequency, binary-lift over the BIT rather than writing a new data structure.

Arrays: Two Sum

```
// Store complements as you scan; check before inserting the current value so an element is not reused.
vector<int> two_sum(const vector<int>& a, int target) {
    unordered_map<int,int> pos;
    for (int i = 0; i < (int)a.size(); ++i) {
        auto it = pos.find(target - a[i]);
        if (it != pos.end()) return {it->second, i};
```

```
        pos[a[i]] = i;
    }
    return {};
}
```

Arrays: 3Sum

```
// Sort, fix one value, and use two pointers; skipping equal values prevents duplicate triples.
vector<vector<int>> three_sum(vector<int> a) {
    sort(a.begin(), a.end());
    vector<vector<int>> ans;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (i && a[i] == a[i-1]) continue;
        int l = i + 1, r = (int)a.size() - 1;
        while (l < r) {
            long long s = 1LL * a[i] + a[l] + a[r];
            if (s < 0) ++l;
            else if (s > 0) --r;
            else {
                ans.push_back({a[i], a[l], a[r]});
                int x = a[l], y = a[r];
                while (l < r && a[l] == x) ++l;
                while (l < r && a[r] == y) --r;
            }
        }
    }
    return ans;
}
```

Arrays: Kadane

```
// Initialize from the first element so all-negative arrays return the largest negative value.
long long kadane(const vector<long long>& a) {
    if (a.empty()) return 0;
    long long cur = a[0], best = a[0];
    for (int i = 1; i < (int)a.size(); ++i) {
        cur = max(a[i], cur + a[i]);
        best = max(best, cur);
    }
    return best;
}
```

Arrays: Next permutation

```
// Find the rightmost ascent, swap with the rightmost larger value, then reverse the suffix.
void next_perm(vector<int>& a) {
    int n = a.size(), i = n - 2;
    while (i >= 0 && a[i] >= a[i+1]) --i;
    if (i < 0) { reverse(a.begin(), a.end()); return; }
    int j = n - 1;
    while (a[j] <= a[i]) --j;
    swap(a[i], a[j]);
    reverse(a.begin() + i + 1, a.end());
}
```

Arrays: Rotated sorted search

```
// One half is sorted at every step; test whether the target lies in that half before discarding it.
int search_rotated(const vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (a[m] == target) return m;
```

```
        if (a[l] <= a[m]) {
            if (a[l] <= target && target < a[m]) r = m - 1;
            else l = m + 1;
        } else {
            if (a[m] < target && target <= a[r]) l = m + 1;
            else r = m - 1;
        }
    }
    return -1;
}
```

Arrays: Trapping Rain Water

```
// Move the side with the smaller boundary because its trapped water is already determined by that side.
long long trap_water(const vector<int>& h) {
    int l = 0, r = (int)h.size() - 1;
    long long lm = 0, rm = 0, ans = 0;
    while (l <= r) {
        if (h[l] <= h[r]) {
            lm = max(lm, (long long)h[l]);
            ans += lm - h[l++];
        } else {
            rm = max(rm, (long long)h[r]);
            ans += rm - h[r--];
        }
    }
    return ans;
}
```

Arrays: Merge intervals

```
// Sorting by start makes all intervals that can overlap the current merged interval adjacent.
vector<vector<int>> merge_intervals(vector<vector<int>> a) {
    if (a.empty()) return {};
    sort(a.begin(), a.end());
    vector<vector<int>> ans{a[0]};
    for (int i = 1; i < (int)a.size(); ++i) {
        if (a[i][0] <= ans.back()[1]) ans.back()[1] = max(ans.back()[1], a[i][1]);
        else ans.push_back(a[i]);
    }
    return ans;
}
```

Search: lower/upper bounds

```
// Use [l,r) and a predicate: lower_bound is first >= x, upper_bound is first > x.
int lower_bound_idx(const vector<int>& a, int x) {
    int l = 0, r = a.size();
    while (l < r) { int m = l + (r-l)/2; if (a[m] < x) l = m + 1; else r = m; }
    return l;
}
int upper_bound_idx(const vector<int>& a, int x) {
    int l = 0, r = a.size();
    while (l < r) { int m = l + (r-l)/2; if (a[m] <= x) l = m + 1; else r = m; }
    return l;
}
```

Search: binary search on the answer

```
// The main change for allocation-style problems is the monotone feasibility predicate; binary-search the
↪  smallest feasible limit.
bool pages_ok(const vector<int>& a, int k, long long lim) {
    int students = 1; long long cur = 0;
```

```
    for (int x : a) {
        if (x > lim) return false;
        if (cur + x <= lim) cur += x;
        else cur = x, ++students;
    }
    return students <= k;
}
long long allocate_pages(const vector<int>& a, int k) {
    if (k > (int)a.size()) return -1;
    long long l = *max_element(a.begin(), a.end()), r = accumulate(a.begin(), a.end(), 0LL);
    while (l < r) { long long m = l + (r-l)/2; if (pages_ok(a,k,m)) r=m; else l=m+1; }
    return l;
}
```

Variant - Aggressive Cows: keep the same binary-search skeleton but make `ok(d)` greedily place cows and maximize the feasible distance. The search direction becomes `ans=d; l=m+1` on success.

Stack/heap: sliding-window maximum

```
// Remove expired indices first, then smaller values from the back; the front is the maximum for the
 ↪  current window.
vector<int> sliding_max(const vector<int>& a, int k) {
    deque<int> dq; vector<int> ans;
    for (int i=0;i<(int)a.size();++i) {
        while (!dq.empty() && dq.front() <= i-k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k-1) ans.push_back(a[dq.front()]);
    }
    return ans;
}
```

Stack: next greater element

```
// Maintain a decreasing stack of indices; when a larger value arrives, it resolves all smaller pending
 ↪  elements.
vector<int> next_greater(const vector<int>& a) {
    int n=a.size(); vector<int> ans(n,-1), st;
    for(int i=n-1;i>=0;--i){
        while(!st.empty() && a[st.back()]<=a[i]) st.pop_back();
        if(!st.empty()) ans[i]=a[st.back()];
        st.push_back(i);
    }
    return ans;
}
```

Hashing: longest unique substring

```
// Store each character's latest index; jump the left boundary past duplicates instead of shrinking one
 ↪  step at a time.
int longest_unique(const string& s) {
    vector<int> last(256,-1);
    int l=0,ans=0;
    for(int r=0;r<(int)s.size();++r){
        l=max(l,last[(unsigned char)s[r]]+1);
        last[(unsigned char)s[r]]=r;
        ans=max(ans,r-l+1);
    }
    return ans;
}
```

Hashing: minimum window

```
// Count required characters and track how many required instances are currently satisfied inside the
 ↪  window.
string min_window(const string& s,const string& t){
    vector<int> need(256),have(256);for(unsigned char c:t)++need[c];
    int req=0;for(int x:need)req+=x>0;int sat=0,l=0,bl=0,br=INT_MAX;
    for(int r=0;r<(int)s.size();++r){unsigned char c=s[r];++have[c];if(need[c]&&have[c]==need[c])++sat;
        while(sat==req&&l<=r){if(r-l<br-bl)bl=l,br=r;unsigned char
         ↪  d=s[l++];--have[d];if(need[d]&&have[d]<need[d])--sat;}
    }
    return br==INT_MAX?"":s.substr(bl,br-bl+1);
}
```

Strings: KMP prefix function

```
// pi[i] is the longest proper prefix that is also a suffix; on mismatch jump to pi[j-1] instead of
 ↪  restarting.
vector<int> prefix_func(const string& s) {
    int n=s.size();vector<int> pi(n);
    for(int i=1;i<n;++i){int j=pi[i-1];while(j&&s[i]!=s[j])j=pi[j-1];pi[i]=j+(s[i]==s[j]);}
    return pi;
}
```

Variants: pattern search uses `pi` on the pattern and jumps `j`; border/period problems use the same array. Z-function is the alternate linear template when the question is phrased around prefix matches at every position.

Strings: Trie

```
// Each node represents a prefix; insertion and lookup both walk characters from the root.
struct Trie {
    struct Node{array<int,26> nxt;bool end=false;Node(){nxt.fill(-1);}};
    vector<Node> t{Node()};
    void insert(const string&s){int v=0;for(char c:s){int
     ↪  x=c-'a';if(t[v].nxt[x]==-1){t[v].nxt[x]=t.size();t.emplace_back();}v=t[v].nxt[x];}t[v].end=true;}
    bool find(const string&s){int v=0;for(char c:s){int x=c-'a';if(t[v].nxt[x]==-1)return
     ↪  false;v=t[v].nxt[x];}return t[v].end;}
};
```

Graph: BFS

```
// BFS is the default shortest-path traversal when every edge has equal cost; -1 marks unreachable
 ↪  vertices.
vector<int> bfs_dist(int s,const vector<vector<int>>& g){
    vector<int> d(g.size(),-1);queue<int>q;q.push(s);d[s]=0;
    while(!q.empty()){int u=q.front();q.pop();for(int v:g[u])if(d[v]<0)d[v]=d[u]+1,q.push(v);}
    return d;
}
```

Graph: Dijkstra

```
// Skip stale heap entries because priority_queue has no decrease-key operation.
vector<long long> dijkstra(int s,const vector<vector<pair<int,int>>>& g){
    int n=g.size();vector<long long>d(n,4e18);priority_queue<pair<long long,int>,vector<pair<long
     ↪  long,int>>,greater<pair<long long,int>>>pq;
    d[s]=0;pq.push({0,s});
    while(!pq.empty()){auto [du,u]=pq.top();pq.pop();if(du!=d[u])continue;for(auto
     ↪  [v,w]:g[u])if(d[v]>du+w)d[v]=du+w,pq.push({d[v],v});}
    return d;
}
```

Graph: 0-1 BFS

```
// For weights 0/1, push zero-weight relaxations to the front and one-weight relaxations to the back.
```

```cpp
vector<int> bfs01(int s,const vector<vector<pair<int,int>>>& g){
    int n=g.size();vector<int>d(n,1e9);deque<int>q;q.push_back(s);d[s]=0;
    while(!q.empty()){int u=q.front();q.pop_front();for(auto
     ↪  [v,w]:g[u])if(d[v]>d[u]+w){d[v]=d[u]+w;if(w)q.push_back(v);else q.push_front(v);}}
    return d;
}
```

Graph: topological sort

```cpp
// Kahn's algorithm removes zero-indegree vertices; if fewer than n vertices are output, the graph
 ↪  contains a cycle.
vector<int> topo_sort(int n,const vector<vector<int>>&g){
    vector<int>in(n),ord;queue<int>q;for(int u=0;u<n;++u)for(int v:g[u])++in[v];
    for(int i=0;i<n;++i)if(!in[i])q.push(i);
    while(!q.empty()){int u=q.front();q.pop();ord.push_back(u);for(int v:g[u])if(!--in[v])q.push(v);}
    return ord.size()==n?ord:vector<int>();
}
```

Connectivity: DSU

```cpp
// Path compression plus union by size makes component merges essentially constant amortized time.
struct DSU {
    vector<int> p,sz;
    DSU(int n):p(n),sz(n,1){iota(p.begin(),p.end(),0);}
    int find(int x){return x==p[x]?x:p[x]=find(p[x]);}
    bool unite(int a,int b){a=find(a);b=find(b);if(a==b)return
     ↪  false;if(sz[a]<sz[b])swap(a,b);p[b]=a;sz[a]+=sz[b];return true;}
};
```

MST: Kruskal

```cpp
// Sort edges by weight and add only edges joining different DSU components; stop after n-1 accepted
 ↪  edges.
struct Edge{int u,v;long long w;};
long long kruskal(int n,vector<Edge> e){
    sort(e.begin(),e.end(),[](auto&a,auto&b){return a.w<b.w;});DSU d(n);long long ans=0;int used=0;
    for(auto x:e)if(d.unite(x.u,x.v)){ans+=x.w;if(++used==n-1)break;}
    return used==n-1?ans:LLONG_MAX;
}
```

Trees: LCA by binary lifting

```cpp
// Precompute 2^j ancestors; lift the deeper node first, then raise both while their ancestors differ.
struct LCA {
    int n,LG;vector<int>dep;vector<vector<int>>up;
    LCA(const vector<vector<int>>&g,int root=0):n(g.size()),LG(1),dep(n),up(){while((1<<LG)<=n)++LG;up.
 ↪  assign(LG,vector<int>(n));dfs(root,root,g);}
    void dfs(int u,int p,const vector<vector<int>>&g){up[0][u]=p;for(int
     ↪  j=1;j<LG;++j)up[j][u]=up[j-1][up[j-1][u]];for(int v:g[u])if(v!=p)dep[v]=dep[u]+1,dfs(v,u,g);}
    int query(int u,int v){if(dep[u]<dep[v])swap(u,v);int d=dep[u]-dep[v];for(int
     ↪  j=0;j<LG;++j)if(d>>j&1)u=up[j][u];if(u==v)return u;for(int
     ↪  j=LG-1;j>=0;--j)if(up[j][u]!=up[j][v])u=up[j][u],v=up[j][v];return up[0][u];}
};
```

Trees: diameter

```cpp
// The farthest vertex from any tree node is an endpoint of a diameter; run two BFS/DFS traversals.
pair<int,int> farthest(int s,const vector<vector<int>>&g){
    queue<int>q;q.push(s);vector<int>d(g.size(),-1);d[s]=0;int best=s;
    while(!q.empty()){int u=q.front();q.pop();for(int
     ↪  v:g[u])if(d[v]<0)d[v]=d[u]+1,q.push(v),best=d[v]>d[best]?v:best;}
    return {best,d[best]};
}
```

```
int tree_diameter(const vector<vector<int>>&g){auto a=farthest(0,g).first;return farthest(a,g).second;}
```

DP: 0/1 knapsack

```
// Reverse capacity iteration prevents the same item from being used more than once.
int knapsack01(const vector<int>& w,const vector<int>& val,int W){
    vector<int>dp(W+1);for(int i=0;i<(int)w.size();++i)for(int
 ↪  c=W;c>=w[i];--c)dp[c]=max(dp[c],dp[c-w[i]]+val[i]);return dp[W];
}
```

Variant: unbounded knapsack uses the same transition but loops `c` upward so the current item can be reused.

DP: subset sum / partition

```
// Reverse the sum loop for 0/1 usage; partition-equal reduces to subset sum on total_sum/2 when the
 ↪  total is even.
bool subset_sum(const vector<int>& a,int S){
    vector<char>dp(S+1);dp[0]=1;
    for(int x:a)for(int s=S;s>=x;--s)dp[s]|=dp[s-x];
    return dp[S];
}
```

DP: LIS

```
// For strict LIS use lower_bound; for non-decreasing LIS replace it with upper_bound.
int lis(const vector<int>& a){
    vector<int>t;for(int x:a){auto it=lower_bound(t.begin(),t.end(),x);if(it==t.end())t.push_back(x);else
 ↪  *it=x;}return t.size();
}
```

DP: LCS

```
// dp[j] is the best LCS length for the processed prefix of the first string and s2[0..j).
int lcs(const string& a,const string& b){
    vector<int>dp(b.size()+1);
    for(char x:a){int prev=0;for(int j=1;j<=(int)b.size();++j){int
     ↪  old=dp[j];if(x==b[j-1])dp[j]=prev+1;else dp[j]=max(dp[j],dp[j-1]);prev=old;}}
    return dp.back();
}
```

DP: edit distance

```
// Keep only the previous row; the three transitions are insert, delete, and replace.
int edit_distance(const string& a,const string& b){
    vector<int>dp(b.size()+1);iota(dp.begin(),dp.end(),0);
    for(int i=1;i<=(int)a.size();++i){vector<int>ndp(b.size()+1);ndp[0]=i;for(int j=1;j<=(int)b.size();++
     ↪  j)ndp[j]=a[i-1]==b[j-1]?dp[j-1]:1+min({dp[j],ndp[j-1],dp[j-1]});dp.swap(ndp);}
    return dp.back();
}
```

DP: Word Break

```
// dp[i] means s[0..i) is segmentable; test every previous cut whose dictionary word ends at i.
bool word_break(const string&s,const unordered_set<string>&dict){
    int n=s.size();vector<char>dp(n+1);dp[0]=1;
    for(int i=1;i<=n;++i)for(int j=0;j<i;++j)if(dp[j]&&dict.count(s.substr(j,i-j))){dp[i]=1;break;}
    return dp[n];
}
```

DP: House Robber

```
// Keep the best result when the current house is skipped versus robbed.
long long house_robber(const vector<long long>& a){
    long long take=0,skip=0;
```

```
    for(long long x:a){long long ntake=skip+x,nskip=max(skip,take);take=ntake;skip=nskip;}
    return max(take,skip);
}
```

DP: Unique paths / minimum path sum

```
// The same grid-state shape supports counting paths or minimizing cost by changing only the transition
 ↪  operator and identity.
long long unique_paths(int m,int n){
    vector<long long>dp(n);dp[0]=1;
    for(int i=0;i<m;++i)for(int j=1;j<n;++j)dp[j]+=dp[j-1];
    return dp[n-1];
}
```

Variant: minimum path sum changes the scalar operation from `+` across paths to `min(previous_top, previous_left) + grid[i][j]`.

Segment Tree mini-bank

Generic associative Segment Tree

```
// Only associativity is required; commutativity is optional, and the identity handles disjoint query
 ↪  pieces.
template<class T,class F>
struct SegTree {
    int n; T id; F op; vector<T> st;
    SegTree(const vector<T>& a,T id,F op):n(a.size()),id(id),op(op),st(2*n,id){
        for(int i=0;i<n;++i)st[n+i]=a[i];
        for(int i=n-1;i;--i)st[i]=op(st[i<<1],st[i<<1|1]);
    }
    void setPoint(int p,T x){
        st[p+=n]=x;
        for(p>>=1;p;p>>=1)st[p]=op(st[p<<1],st[p<<1|1]);
    }
    T query(int l,int r){
        T L=id,R=id;
        for(l+=n,r+=n;l<r;l>>=1,r>>=1){
            if(l&1)L=op(L,st[l++]);
            if(r&1)R=op(st[--r],R);
        }
        return op(L,R);
    }
};
```

What changes: `op` + `id` only. Examples: sum/0, xor/0, gcd/0, min/+INF, max/-INF. For non-commutative operations, keep the left accumulator and right accumulator separate so query order is preserved.

Generic Lazy Segment Tree

```
// Range updates need three pieces: merge child nodes, apply a tag to one node, and compose old/new tags
 ↪  in order.
template<class Node,class Tag,class Merge,class Apply,class Compose>
struct LazySegTree {
    int n; Node idNode; Tag idTag; Merge merge; Apply apply; Compose compose;
    vector<Node> st; vector<Tag> lazy; vector<char> has;
    LazySegTree(const vector<Node>&a,Node idNode,Tag idTag,Merge merge,Apply apply,Compose
 ↪  compose):n(a.size()),idNode(idNode),idTag(idTag),merge(merge),apply(apply),compose(compose),st(4*n,
 ↪  idNode),lazy(4*n,idTag),has(4*n){
        build(1,0,n-1,a);
    }
    void build(int v,int l,int r,const vector<Node>&a){
        if(l==r){st[v]=a[l];return;}
        int m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);st[v]=merge(st[v*2],st[v*2+1]);
    }
```

```
    void applyTag(int v,const Tag&t,int len){apply(st[v],t,len);if(has[v])lazy[v]=compose(t,lazy[v]);else
     ↪ lazy[v]=t,has[v]=1;}
    void push(int v,int l,int r){if(!has[v]||l==r)return;int
     ↪ m=(l+r)/2;applyTag(v*2,lazy[v],m-l+1);applyTag(v*2+1,lazy[v],r-m);lazy[v]=idTag;has[v]=0;}
    void update(int ql,int qr,const Tag&t,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return;if(ql<=l&&r<=qr){applyTag(v,t,r-l+1);return;}
        push(v,l,r);int
 ↪ m=(l+r)/2;update(ql,qr,t,v*2,l,m);update(ql,qr,t,v*2+1,m+1,r);st[v]=merge(st[v*2],st[v*2+1]);
    }
    Node query(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return idNode;if(ql<=l&&r<=qr)return st[v];
        push(v,l,r);int m=(l+r)/2;return merge(query(ql,qr,v*2,l,m),query(ql,qr,v*2+1,m+1,r));
    }
};
```

Lazy Segment Tree: range add + sum/min/max

```
// A range add shifts sum, minimum, and maximum predictably, so one additive tag is enough for all three.
struct AddMinMax {
    struct Node{long long sum,mn,mx;int len;};
    int n; vector<Node> st; vector<long long> lz;
    AddMinMax(const vector<long long>& a):n(a.size()),st(4*n),lz(4*n){build(1,0,n-1,a);}
    void build(int v,int l,int r,const vector<long long>& a){
        if(l==r){st[v]={a[l],a[l],a[l],1};return;}int
         ↪ m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);pull(v);
    }
    void pull(int v){st[v].sum=st[v*2].sum+st[v*2+1].sum;st[v].mn=min(st[v*2].mn,st[v*2+1].mn);st[v].mx=
     ↪ max(st[v*2].mx,st[v*2+1].mx);st[v].len=st[v*2].len+st[v*2+1].len;}
    void apply(int v,long long x){st[v].sum+=x*st[v].len;st[v].mn+=x;st[v].mx+=x;lz[v]+=x;}
    void push(int v){if(lz[v]){apply(v*2,lz[v]);apply(v*2+1,lz[v]);lz[v]=0;}}
    void add(int ql,int qr,long long x,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return;if(ql<=l&&r<=qr){apply(v,x);return;}push(v);int
         ↪ m=(l+r)/2;add(ql,qr,x,v*2,l,m);add(ql,qr,x,v*2+1,m+1,r);pull(v);
    }
    Node query(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return {0,LLONG_MAX,LLONG_MIN,0};
        if(ql<=l&&r<=qr)return st[v];push(v);int m=(l+r)/2;auto
         ↪ a=query(ql,qr,v*2,l,m),b=query(ql,qr,v*2+1,m+1,r);if(!a.len)return b;if(!b.len)return
         ↪ a;return {a.sum+b.sum,min(a.mn,b.mn),max(a.mx,b.mx),a.len+b.len};
    }
};
```

Variant: range add + sum needs only `{sum,len}`. Range add + min/max needs `{mn,mx}`. The lazy operation remains `x += add`.

Segment Tree: maximum subarray

```
// Store sum, best prefix, best suffix, and best subarray; these four fields are closed under
 ↪ concatenation.
struct MaxSubSeg {
    struct N{long long sum,pref,suff,best;};
    int n; vector<N> st;
    N mergeN(N a,N b){return
 ↪ {a.sum+b.sum,max(a.pref,a.sum+b.pref),max(b.suff,b.sum+a.suff),max({a.best,b.best,a.suff+b.pref})};}
    MaxSubSeg(const vector<long long>& a):n(a.size()),st(4*n){build(1,0,n-1,a);}
    void build(int v,int l,int r,const vector<long long>&
     ↪ a){if(l==r){st[v]={a[l],a[l],a[l],a[l]};return;}int
     ↪ m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);st[v]=mergeN(st[v*2],st[v*2+1]);}
    void setPoint(int p,long long x,int v=1,int l=0,int
     ↪ r=-1){if(r<0)r=n-1;if(l==r){st[v]={x,x,x,x};return;}int
     ↪ m=(l+r)/2;if(p<=m)setPoint(p,x,v*2,l,m);else
     ↪ setPoint(p,x,v*2+1,m+1,r);st[v]=mergeN(st[v*2],st[v*2+1]);}
};
```

Segment Tree: minimum + frequency

```
// Equal minima add their frequencies; otherwise keep the node with the smaller minimum.
struct MinCountSeg {
    using N=pair<int,int>;
    int n; vector<N> st;
    static N op(N a,N b){if(a.first<b.first)return a;if(b.first<a.first)return b;return
     ↪ {a.first,a.second+b.second};}
    MinCountSeg(const vector<int>& a):n(a.size()),st(2*n){for(int i=0;i<n;++i)st[n+i]={a[i],1};for(int
 ↪ i=n-1;i;--i)st[i]=op(st[i<<1],st[i<<1|1]);}
    N query(int l,int r){N L={INT_MAX,0},R={INT_MAX,0};for(l+=n,r+=n;l<r;l>>=1,r>>=1){if(l&1)L=op(L,st[l
 ↪ ++]);if(r&1)R=op(st[--r],R);}return op(L,R);}
};
```

CP-Algorithms recursive Segment Tree pack - memorize this, adapt the merge

Convention: this pack follows the CP-Algorithms recursive style and uses half-open ranges `[tl,tr)` and query ranges `[l,r)`. A point `p` is a leaf when `tr-tl==1`. Do not mix this with the inclusive `[l,r]` variants elsewhere in this file. Standard build/query/update are `O(n) / O(log n) / O(log n)`.

How to call the recursive functions (memorize this before the variants) This pack uses **half-open ranges** `[tl,tr)` and query/update ranges `[l,r)`. Thus `[1,4)` contains indexes `1,2,3`; `l==r` is empty. This matches the current CP-Algorithms implementation convention.

Name	Meaning	Typical value / rule
`n`	Number of elements in the original array	`n = a.size()`; it is **not** the tree-vector size
`t` / `lazy`	Stored node summaries / pending tags	Recursive trees allocate `4 * max(1,n)`; `4*n` is storage only, never a query boundary
`v`	Index of the current node inside `t`	Root is `1`; children are `2*v` and `2*v+1`
`tl,tr`	Segment represented by node `v`	Start root with `v=1, tl=0, tr=n`; current node covers `[tl,tr)`
`l,r`	Requested range for query/update	Pass your problem's zero-based `[l,r)`; **not** the current node bounds
`tm`	Midpoint of the current node segment	`tm=(tl+tr)/2`; recurse `[tl,tm)` and `[tm,tr)`
`p`	One point index for a point update	`0 <= p < n`; go left if `p < tm`, otherwise right
`x` / `tag`	New value or range-update payload	Meaning depends on the structure: assign value, add delta, `chmin` cap, or lazy tag

The internal recursive call uses both kinds of boundaries. Example: `query(v,tl,tr,l,r)` means "look at node `v`, whose interval is `[tl,tr)`, and answer only the requested intersection with `[l,r)`." You normally should not manually pass `v/tl/tr`: call the public wrappers below.

```
vector<long long> a = {5, 1, 7, 3, 2};
SumSeg sum(a);                      // recursively builds root (1,0,n)
long long s = sum.query(1, 4);      // a[1]+a[2]+a[3]
sum.setPoint(2, 10);                // assign a[2]=10

LazySumSeg lazySum(a);
lazySum.add(1, 4, 5);               // add 5 to a[1], a[2], a[3]
long long s2 = lazySum.query(0, 4);

MinCountSeg mc(a); auto mn = mc.query(1, 4); // mn.mn and mn.cnt
MaxSubSeg ms(a); auto best = ms.query(0, 5);   // best.ans
AddStatsSeg stats(a); auto q = stats.query(1, 4); // q.sum, q.mn, q.mx
AssignSumSeg as(a); as.assignRange(1, 4, 8); as.addRange(0, 2, 3);
AffineSumSeg af(a); af.update(1, 4, {2, 3}); // x -> 2*x+3, then query(1,4)
SegBeats beats(a); beats.chmin(1, 4, 6); beats.querySum(0, 5); beats.queryMax(0, 5);
```

```
MergeSortTree mst(vector<int>{5,1,7,3,2}); int cnt = mst.countLE(1, 4, 3);
PersistentSeg pst; pst.build(vector<int>{5,1,7,3,2}); int kth = pst.kth(1, 4, 2); // k is 1-based
```

Which object should I create for each variant?

Need	Structure	Public operations	What actually changes
Sum/min/max/GCD/XOR/OR/AND/product/matrix/string aggregate + point assignment	SumSeg as starting template (rename/generalize stored type and +)	`query(l,r)`, `setPoint(p,x)`	Store a node type; replace merge and disjoint-query identity. Use `gcd` with identity `0`, min with `LLONG_MAX`, max with `LLONG_MIN`, XOR with `0`, product with `1`; keep left/right order for non-commutative merges.
Range minimum + frequency	`MinCountSeg`	`query(l,r)`, `setPoint(p,x)`	Node `{mn,cnt,empty}`; take smaller minimum, or add counts on ties.
Maximum **non-empty** subarray sum	`MaxSubSeg`	`query(l,r)`, `setPoint(p,x)`	Node `{sum,pref,suff,ans,empty}`; crossing answer is `left.suff + right.pref`.
Generic range update + range query	`GenericLazySeg<Node,Tag,...>`	`update(l,r,tag)`, `query(l,r)`	Supply `merge`, `apply(node,tag,len)`, and `compose(newTag,oldTag)`. New tag is applied after old tag.
Range add + sum	`LazySumSeg`	`add(l,r,x)`, `query(l,r)`	Node is sum; apply `sum += x*(tr-tl)`; lazy tags add.
Range add + sum/min/max	`AddStatsSeg`	`add(l,r,x)`, `query(l,r)`	Node `{sum,mn,mx,empty}`; add shifts all statistics predictably.
Range assignment + range add + sum	`AssignSumSeg`	`assignRange(l,r,x)`, `addRange(l,r,x)`, `query(l,r)`	Keep `hasSet`; assignment overrides older tags; push assignment before add.
Affine range update + sum	`AffineSumSeg`	`update(l,r,{a,b})`, `query(l,r)`	Apply `x -> a*x+b`; compose new `(a,b)` after old `(c,d)` as `(a*c,a*d+b)`.
Range `chmin` + sum/max	`SegBeats`	`chmin(l,r,x)`, `querySum(l,r)`, `queryMax(l,r)`	Special Beats condition `secondMax < x < max`; ordinary lazy add tags are not enough.
Count values `<=x` in a static range	`MergeSortTree`	`countLE(l,r,x)`	Each node stores a sorted vector; no efficient ordinary point-update support.
k-th smallest in a subarray	`PersistentSeg`	`build(a)`, `kth(l,r,k)`	Keep prefix roots; `k` is one-based and range `[l,r)` is zero-based/half-open.
GCD DP, non-adjacent DP, fixed-point DP	Problem-specific node/DP	See the corresponding problem card	Reuse the recursive tree mechanics; customize the node and merge, or `max` + point-`chmax`.

When to use `4*n` versus `n`: recursive `v*2` trees need `4*max(1,n)` slots because node indexes are heap-style and not all array slots are used. The iterative generic tree in the earlier mini-bank stores leaves at `st[n..2*n)` and therefore allocates `2*n`. Do not pass `4*n` as `tr`; `tr` is the array interval endpoint, `n`.

1. Canonical CP-Algorithms base: range sum + point assignment

```
struct SumSeg {
    int n;
```

```
    vector<long long> t;
    SumSeg(const vector<long long>& a) : n((int)a.size()), t(4 * max(1, n)) {
        if (n) build(a, 1, 0, n);
    }
    void build(const vector<long long>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { t[v] = a[tl]; return; }
        int tm = (tl + tr) / 2;
        build(a, v * 2, tl, tm);
        build(a, v * 2 + 1, tm, tr);
        t[v] = t[v * 2] + t[v * 2 + 1];
    }
    long long query(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return 0;
        if (l <= tl && tr <= r) return t[v];
        int tm = (tl + tr) / 2;
        return query(v * 2, tl, tm, l, r) + query(v * 2 + 1, tm, tr, l, r);
    }
    void setPoint(int v, int tl, int tr, int p, long long x) {
        if (tr - tl == 1) { t[v] = x; return; }
        int tm = (tl + tr) / 2;
        if (p < tm) setPoint(v * 2, tl, tm, p, x);
        else setPoint(v * 2 + 1, tm, tr, p, x);
        t[v] = t[v * 2] + t[v * 2 + 1];
    }
    long long query(int l, int r) { return l >= r ? 0 : query(1, 0, n, l, r); }
    void setPoint(int p, long long x) { setPoint(1, 0, n, p, x); }
};
```

2. Ordinary range-query variants: change only `merge` and the identity

Need	Store / merge	Identity for a disjoint query
Sum	`long long`, `a+b`	`0`
Minimum	`long long`, `min(a,b)`	`LLONG_MAX`
Maximum	`long long`, `max(a,b)`	`LLONG_MIN`
GCD	`gcd(a,b)`	`0`
XOR / OR	`a^b` / `a|b`	`0`
AND	`a&b`	all bits set, e.g. `~0LL`
Product	`a*b` (watch overflow/modulo)	`1`
Matrix product / string concat	operation in original left-to-right order	identity matrix / empty string

For a generic monoid tree, make `t` store the node type, replace the `+` in `build` and `setPoint` with `merge(left,right)`, and make the no-overlap branch of `query` return `identity`. **For non-commutative merges, preserve left-before-right order.** GCD Segment Tree cards only need `merge = gcd` and identity `0` before adding their problem-specific DP logic.

3. Minimum value + frequency

```
struct MinCountSeg {
    struct Node { long long mn; int cnt; bool empty; };
    int n; vector<Node> t;
    static Node mergeN(Node a, Node b) {
        if (a.empty) return b;
        if (b.empty) return a;
        if (a.mn < b.mn) return a;
        if (b.mn < a.mn) return b;
        return {a.mn, a.cnt + b.cnt, false};
    }
    MinCountSeg(const vector<long long>& a) : n((int)a.size()), t(4 * max(1, n)) {
        if (n) build(a, 1, 0, n);
    }
    void build(const vector<long long>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { t[v] = {a[tl], 1, false}; return; }
```

```
        int tm = (tl + tr) / 2;
        build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr);
        t[v] = mergeN(t[v * 2], t[v * 2 + 1]);
    }
    Node query(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return {0, 0, true};
        if (l <= tl && tr <= r) return t[v];
        int tm = (tl + tr) / 2;
        return mergeN(query(v * 2, tl, tm, l, r), query(v * 2 + 1, tm, tr, l, r));
    }
    void setPoint(int v, int tl, int tr, int p, long long x) {
        if (tr - tl == 1) { t[v] = {x, 1, false}; return; }
        int tm = (tl + tr) / 2;
        if (p < tm) setPoint(v * 2, tl, tm, p, x); else setPoint(v * 2 + 1, tm, tr, p, x);
        t[v] = mergeN(t[v * 2], t[v * 2 + 1]);
    }
    Node query(int l, int r) { return query(1, 0, n, l, r); }
    void setPoint(int p, long long x) { setPoint(1, 0, n, p, x); }
};
```

4. Maximum subarray sum Each node stores `sum`, best non-empty `pref`, best non-empty `suff`, and best non-empty `ans`. The `empty` flag gives a valid identity for disjoint query parts.

```
struct MaxSubSeg {
    struct Node { long long sum, pref, suff, ans; bool empty; };
    int n; vector<Node> t;
    static Node mergeN(Node a, Node b) {
        if (a.empty) return b;
        if (b.empty) return a;
        return {a.sum + b.sum, max(a.pref, a.sum + b.pref),
                max(b.suff, b.sum + a.suff), max({a.ans, b.ans, a.suff + b.pref}), false};
    }
    MaxSubSeg(const vector<long long>& a) : n((int)a.size()), t(4 * max(1, n)) {
        if (n) build(a, 1, 0, n);
    }
    void build(const vector<long long>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { long long x = a[tl]; t[v] = {x, x, x, x, false}; return; }
        int tm = (tl + tr) / 2;
        build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr);
        t[v] = mergeN(t[v * 2], t[v * 2 + 1]);
    }
    Node query(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return {0, 0, 0, 0, true};
        if (l <= tl && tr <= r) return t[v];
        int tm = (tl + tr) / 2;
        return mergeN(query(v * 2, tl, tm, l, r), query(v * 2 + 1, tm, tr, l, r));
    }
    void setPoint(int v, int tl, int tr, int p, long long x) {
        if (tr - tl == 1) { t[v] = {x, x, x, x, false}; return; }
        int tm = (tl + tr) / 2;
        if (p < tm) setPoint(v * 2, tl, tm, p, x); else setPoint(v * 2 + 1, tm, tr, p, x);
        t[v] = mergeN(t[v * 2], t[v * 2 + 1]);
    }
    Node query(int l, int r) { return query(1, 0, n, l, r); }
    void setPoint(int p, long long x) { setPoint(1, 0, n, p, x); }
};
```

5. Generic lazy Segment Tree in CP recursive style This is the reusable lazy skeleton. `apply(node, tag, len)` mutates a node, and `compose(newTag, oldTag)` returns the tag equivalent to applying the old tag and then the new one.

```
template<class Node, class Tag, class Merge, class Apply, class Compose>
struct GenericLazySeg {
    int n; Node idNode; Tag idTag; Merge merge; Apply apply; Compose compose;
    vector<Node> t; vector<Tag> lazy; vector<char> has;
```

```
    GenericLazySeg(const vector<Node>& a, Node idN, Tag idT, Merge mg, Apply ap, Compose cp)
        : n((int)a.size()), idNode(idN), idTag(idT), merge(mg), apply(ap), compose(cp),
          t(4 * max(1, n), idNode), lazy(4 * max(1, n), idTag), has(4 * max(1, n), false) {
        if (n) build(a, 1, 0, n);
    }
    void build(const vector<Node>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { t[v] = a[tl]; return; }
        int tm = (tl + tr) / 2;
        build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr); t[v] = merge(t[v * 2], t[v * 2 + 1]);
    }
    void applyTag(int v, const Tag& x, int len) {
        apply(t[v], x, len);
        if (has[v]) lazy[v] = compose(x, lazy[v]); else { lazy[v] = x; has[v] = true; }
    }
    void push(int v, int tl, int tr) {
        if (!has[v] || tr - tl == 1) return;
        int tm = (tl + tr) / 2;
        applyTag(v * 2, lazy[v], tm - tl); applyTag(v * 2 + 1, lazy[v], tr - tm);
        lazy[v] = idTag; has[v] = false;
    }
    void update(int v, int tl, int tr, int l, int r, const Tag& x) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) { applyTag(v, x, tr - tl); return; }
        push(v, tl, tr); int tm = (tl + tr) / 2;
        update(v * 2, tl, tm, l, r, x); update(v * 2 + 1, tm, tr, l, r, x);
        t[v] = merge(t[v * 2], t[v * 2 + 1]);
    }
    Node query(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return idNode;
        if (l <= tl && tr <= r) return t[v];
        push(v, tl, tr); int tm = (tl + tr) / 2;
        return merge(query(v * 2, tl, tm, l, r), query(v * 2 + 1, tm, tr, l, r));
    }
    void update(int l, int r, const Tag& x) { if (l < r) update(1, 0, n, l, r, x); }
    Node query(int l, int r) { return query(1, 0, n, l, r); }
};
```

6. Lazy propagation: range add + range sum Memorize the four operations: `build`, `apply`, `push`, and `pull`. Push before descending; pull after recursive updates. A covered segment `[tl,tr)` has length `tr-tl`.

```
struct LazySumSeg {
    int n; vector<long long> t, lazy;
    LazySumSeg(const vector<long long>& a) : n((int)a.size()), t(4 * max(1, n)), lazy(4 * max(1, n)) {
        if (n) build(a, 1, 0, n);
    }
    void build(const vector<long long>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { t[v] = a[tl]; return; }
        int tm = (tl + tr) / 2;
        build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr); pull(v);
    }
    void pull(int v) { t[v] = t[v * 2] + t[v * 2 + 1]; }
    void apply(int v, int tl, int tr, long long x) { t[v] += x * (tr - tl); lazy[v] += x; }
    void push(int v, int tl, int tr) {
        if (!lazy[v] || tr - tl == 1) return;
        int tm = (tl + tr) / 2;
        apply(v * 2, tl, tm, lazy[v]); apply(v * 2 + 1, tm, tr, lazy[v]); lazy[v] = 0;
    }
    void add(int v, int tl, int tr, int l, int r, long long x) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) { apply(v, tl, tr, x); return; }
        push(v, tl, tr); int tm = (tl + tr) / 2;
        add(v * 2, tl, tm, l, r, x); add(v * 2 + 1, tm, tr, l, r, x); pull(v);
    }
```

```
    long long query(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return 0;
        if (l <= tl && tr <= r) return t[v];
        push(v, tl, tr); int tm = (tl + tr) / 2;
        return query(v * 2, tl, tm, l, r) + query(v * 2 + 1, tm, tr, l, r);
    }
    void add(int l, int r, long long x) { if (l < r) add(1, 0, n, l, r, x); }
    long long query(int l, int r) { return l < r ? query(1, 0, n, l, r) : 0; }
};
```

7. Range add + sum/min/max This is the concrete change when a lazy node must support all three statistics, not just sum.

```
struct AddStatsSeg {
    struct Node { long long sum, mn, mx; bool empty; };
    int n; vector<Node> t; vector<long long> lazy;
    static Node mergeN(Node a, Node b) {
        if (a.empty) return b;
        if (b.empty) return a;
        return {a.sum + b.sum, min(a.mn, b.mn), max(a.mx, b.mx), false};
    }
    AddStatsSeg(const vector<long long>& a) : n((int)a.size()), t(4 * max(1, n)), lazy(4 * max(1, n)) {
        if (n) build(a, 1, 0, n);
    }
    void build(const vector<long long>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { t[v] = {a[tl], a[tl], a[tl], false}; return; }
        int tm = (tl + tr) / 2;
        build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr); pull(v);
    }
    void pull(int v) { t[v] = mergeN(t[v * 2], t[v * 2 + 1]); }
    void apply(int v, int tl, int tr, long long x) {
        t[v].sum += x * (tr - tl); t[v].mn += x; t[v].mx += x; lazy[v] += x;
    }
    void push(int v, int tl, int tr) {
        if (!lazy[v] || tr - tl == 1) return;
        int tm = (tl + tr) / 2;
        apply(v * 2, tl, tm, lazy[v]); apply(v * 2 + 1, tm, tr, lazy[v]); lazy[v] = 0;
    }
    void add(int v, int tl, int tr, int l, int r, long long x) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) { apply(v, tl, tr, x); return; }
        push(v, tl, tr); int tm = (tl + tr) / 2;
        add(v * 2, tl, tm, l, r, x); add(v * 2 + 1, tm, tr, l, r, x); pull(v);
    }
    Node query(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return {0, 0, 0, true};
        if (l <= tl && tr <= r) return t[v];
        push(v, tl, tr); int tm = (tl + tr) / 2;
        return mergeN(query(v * 2, tl, tm, l, r), query(v * 2 + 1, tm, tr, l, r));
    }
    void add(int l, int r, long long x) { if (l < r) add(1, 0, n, l, r, x); }
    Node query(int l, int r) { return query(1, 0, n, l, r); }
};
```

8. Lazy variants - exact changes to the same recursive skeleton

Variant	Node merge / `pull`	Tag application and composition
Range add + sum	`sum = left.sum + right.sum`	`sum += x * len`; compose additions as `old + x`
Range add + minimum	`mn = min(left.mn,right.mn)`	`mn += x`; identity for no overlap is `+INF`
Range add + maximum	`mx = max(left.mx,right.mx)`	`mx += x`; identity for no overlap is `-INF`

Variant	Node merge / `pull`	Tag application and composition
Range add + sum/min/max	Store `{sum,mn,mx}`; merge each field	`sum += x*len`, `mn += x`, `mx += x`; one additive tag works for all fields
Range assignment + sum	Merge sums	Set: `sum=x*len`, pending assignment becomes `x`, pending add resets to `0`; a later set overrides earlier tags
Range add + assignment + sum	Merge sums	Add changes `sum += x*len`; if assignment is pending, increment its value, else accumulate lazy add. Push assignment before add
Affine update `x -> a*x+b` + sum	Merge sums	Apply: `sum = a*sum+b*len`; compose new `(a,b)` after old `(c,d)` as `(a*c, a*d+b)`; identity tag `(1,0)`

For a truly generic lazy tree, define `merge(left,right)`, `apply(node,tag,len)`, and `compose(newTag,oldTag)`. **Composition is chronological: the new tag is applied after the old tag.** That order matters for assignment and affine transformations. If operating modulo `MOD`, normalize every tag/value operation under the modulus.

Full lazy code: range assignment + range sum

```
struct AssignSumSeg {
    int n; vector<long long> sum, addTag, setVal; vector<char> hasSet;
    AssignSumSeg(const vector<long long>& a) : n((int)a.size()), sum(4 * max(1, n)), addTag(4 * max(1,
 ↪  n)), setVal(4 * max(1, n)), hasSet(4 * max(1, n)) { if (n) build(a, 1, 0, n); }
    void build(const vector<long long>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { sum[v] = a[tl]; return; }
        int tm = (tl + tr) / 2; build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr); pull(v);
    }
    void pull(int v) { sum[v] = sum[v * 2] + sum[v * 2 + 1]; }
    void applySet(int v, int tl, int tr, long long x) { sum[v] = x * (tr - tl); setVal[v] = x; hasSet[v]
     ↪  = true; addTag[v] = 0; }
    void applyAdd(int v, int tl, int tr, long long x) { sum[v] += x * (tr - tl); if (hasSet[v]) setVal[v]
     ↪  += x; else addTag[v] += x; }
    void push(int v, int tl, int tr) {
        if (tr - tl == 1) return;
        int tm = (tl + tr) / 2;
        if (hasSet[v]) { applySet(v * 2, tl, tm, setVal[v]); applySet(v * 2 + 1, tm, tr, setVal[v]);
         ↪  hasSet[v] = false; }
        if (addTag[v]) { applyAdd(v * 2, tl, tm, addTag[v]); applyAdd(v * 2 + 1, tm, tr, addTag[v]);
         ↪  addTag[v] = 0; }
    }
    void update(int v, int tl, int tr, int l, int r, long long x, bool isSet) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) { if (isSet) applySet(v, tl, tr, x); else applyAdd(v, tl, tr, x); return;
         ↪  }
        push(v, tl, tr); int tm = (tl + tr) / 2;
        update(v * 2, tl, tm, l, r, x, isSet); update(v * 2 + 1, tm, tr, l, r, x, isSet); pull(v);
    }
    long long query(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return 0;
        if (l <= tl && tr <= r) return sum[v];
        push(v, tl, tr); int tm = (tl + tr) / 2;
        return query(v * 2, tl, tm, l, r) + query(v * 2 + 1, tm, tr, l, r);
    }
    void assignRange(int l, int r, long long x) { if (l < r) update(1, 0, n, l, r, x, true); }
    void addRange(int l, int r, long long x) { if (l < r) update(1, 0, n, l, r, x, false); }
    long long query(int l, int r) { return query(1, 0, n, l, r); }
};
```

Full lazy code: affine update + range sum The affine update is `x -> a*x+b`. The tag composition below applies the incoming tag after the stored one.

```cpp
struct AffineSumSeg {
    struct Tag { long long a = 1, b = 0; };
    int n; vector<long long> sum; vector<Tag> lazy; vector<char> has;
    AffineSumSeg(const vector<long long>& a) : n((int)a.size()), sum(4 * max(1, n)), lazy(4 * max(1, n)),
    ↪ has(4 * max(1, n)) { if (n) build(a, 1, 0, n); }
    void build(const vector<long long>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { sum[v] = a[tl]; return; }
        int tm = (tl + tr) / 2; build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr); pull(v);
    }
    void pull(int v) { sum[v] = sum[v * 2] + sum[v * 2 + 1]; }
    void apply(int v, int tl, int tr, Tag x) {
        sum[v] = x.a * sum[v] + x.b * (tr - tl);
        lazy[v] = {x.a * lazy[v].a, x.a * lazy[v].b + x.b}; has[v] = true;
    }
    void push(int v, int tl, int tr) {
        if (!has[v] || tr - tl == 1) return;
        int tm = (tl + tr) / 2; apply(v * 2, tl, tm, lazy[v]); apply(v * 2 + 1, tm, tr, lazy[v]);
        lazy[v] = {1, 0}; has[v] = false;
    }
    void update(int v, int tl, int tr, int l, int r, Tag x) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) { apply(v, tl, tr, x); return; }
        push(v, tl, tr); int tm = (tl + tr) / 2;
        update(v * 2, tl, tm, l, r, x); update(v * 2 + 1, tm, tr, l, r, x); pull(v);
    }
    long long query(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return 0;
        if (l <= tl && tr <= r) return sum[v];
        push(v, tl, tr); int tm = (tl + tr) / 2;
        return query(v * 2, tl, tm, l, r) + query(v * 2 + 1, tm, tr, l, r);
    }
    void update(int l, int r, Tag x) { if (l < r) update(1, 0, n, l, r, x); }
    long long query(int l, int r) { return query(1, 0, n, l, r); }
};
```

9. Segment Tree Beats: range chmin + range sum/max This is **not** ordinary lazy propagation. Store maximum `mx`, second-largest value `sx`, count of maximum `cnt`, and sum. The fast full-cover update is legal only when `sx < x < mx`.

```cpp
struct SegBeats {
    struct Node { long long mx, sx, sum; int cnt; };
    int n; vector<Node> t;
    SegBeats(const vector<long long>& a) : n((int)a.size()), t(4 * max(1, n)) { if (n) build(a, 1, 0, n);
    ↪ }
    static Node mergeN(Node a, Node b) {
        Node z; z.sum = a.sum + b.sum;
        if (a.mx == b.mx) z.mx = a.mx, z.cnt = a.cnt + b.cnt, z.sx = max(a.sx, b.sx);
        else if (a.mx > b.mx) z.mx = a.mx, z.cnt = a.cnt, z.sx = max(a.sx, b.mx);
        else z.mx = b.mx, z.cnt = b.cnt, z.sx = max(a.mx, b.sx);
        return z;
    }
    void build(const vector<long long>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { t[v] = {a[tl], LLONG_MIN, a[tl], 1}; return; }
        int tm = (tl + tr) / 2; build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr);
        t[v] = mergeN(t[v * 2], t[v * 2 + 1]);
    }
    void apply(int v, long long x) {
        if (x >= t[v].mx) return;
        t[v].sum -= (t[v].mx - x) * t[v].cnt; t[v].mx = x;
    }
    void push(int v, int tl, int tr) {
        if (tr - tl == 1) return;
        apply(v * 2, t[v].mx); apply(v * 2 + 1, t[v].mx);
    }
```

```
    void chmin(int v, int tl, int tr, int l, int r, long long x) {
        if (r <= tl || tr <= l || x >= t[v].mx) return;
        if (l <= tl && tr <= r && x > t[v].sx) { apply(v, x); return; }
        if (tr - tl == 1) { apply(v, x); return; }
        push(v, tl, tr); int tm = (tl + tr) / 2;
        chmin(v * 2, tl, tm, l, r, x); chmin(v * 2 + 1, tm, tr, l, r, x);
        t[v] = mergeN(t[v * 2], t[v * 2 + 1]);
    }
    long long querySum(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return 0;
        if (l <= tl && tr <= r) return t[v].sum;
        push(v, tl, tr); int tm = (tl + tr) / 2;
        return querySum(v * 2, tl, tm, l, r) + querySum(v * 2 + 1, tm, tr, l, r);
    }
    long long queryMax(int v, int tl, int tr, int l, int r) {
        if (r <= tl || tr <= l) return LLONG_MIN;
        if (l <= tl && tr <= r) return t[v].mx;
        push(v, tl, tr); int tm = (tl + tr) / 2;
        return max(queryMax(v * 2, tl, tm, l, r), queryMax(v * 2 + 1, tm, tr, l, r));
    }
    void chmin(int l, int r, long long x) { if (l < r) chmin(1, 0, n, l, r, x); }
    long long querySum(int l, int r) { return querySum(1, 0, n, l, r); }
    long long queryMax(int l, int r) { return queryMax(1, 0, n, l, r); }
};
```

10. Static Merge Sort Tree: count values <= x in [l,r) At each node store a sorted vector instead of a scalar. It is static; the query counts values `<= x` in `[l,r)`.

```
struct MergeSortTree {
    int n; vector<vector<int>> t;
    MergeSortTree(const vector<int>& a) : n((int)a.size()), t(4 * max(1, n)) { if (n) build(a, 1, 0, n);
 ↪  }
    void build(const vector<int>& a, int v, int tl, int tr) {
        if (tr - tl == 1) { t[v] = {a[tl]}; return; }
        int tm = (tl + tr) / 2; build(a, v * 2, tl, tm); build(a, v * 2 + 1, tm, tr);
        merge(t[v * 2].begin(), t[v * 2].end(), t[v * 2 + 1].begin(), t[v * 2 + 1].end(),
 ↪  back_inserter(t[v]));
    }
    int countLE(int v, int tl, int tr, int l, int r, int x) const {
        if (r <= tl || tr <= l) return 0;
        if (l <= tl && tr <= r) return upper_bound(t[v].begin(), t[v].end(), x) - t[v].begin();
        int tm = (tl + tr) / 2;
        return countLE(v * 2, tl, tm, l, r, x) + countLE(v * 2 + 1, tm, tr, l, r, x);
    }
    int countLE(int l, int r, int x) const { return countLE(1, 0, n, l, r, x); }
};
```

A node vector can be updated only by rebuilding its sorted contents; use a different structure if frequent point updates are required.

11. Persistent Segment Tree: k-th value in a subarray Keep a root for each prefix. Inserting one value clones only its root-to-leaf path; previous versions are unchanged. Values are coordinate-compressed and `k` is 1-indexed.

```
struct PersistentSeg {
    struct Node { int left = 0, right = 0, cnt = 0; };
    vector<Node> t{{}};
    vector<int> roots, vals;
    int U = 0;
    int add(int old, int tl, int tr, int p) {
        int v = (int)t.size(); t.push_back(t[old]); ++t[v].cnt;
        if (tr - tl > 1) {
            int tm = (tl + tr) / 2;
            if (p < tm) t[v].left = add(t[old].left, tl, tm, p);
            else t[v].right = add(t[old].right, tm, tr, p);
```

```
        }
        return v;
    }
    void build(const vector<int>& a) {
        vals = a; sort(vals.begin(), vals.end()); vals.erase(unique(vals.begin(), vals.end()),
↪   vals.end());
        U = (int)vals.size(); roots.assign(a.size() + 1, 0);
        for (int i = 0; i < (int)a.size(); ++i) {
            int p = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
            roots[i + 1] = add(roots[i], 0, U, p);
        }
    }
    int kthIndex(int leftRoot, int rightRoot, int tl, int tr, int k) const {
        if (tr - tl == 1) return tl;
        int tm = (tl + tr) / 2;
        int leftCount = t[t[rightRoot].left].cnt - t[t[leftRoot].left].cnt;
        if (k <= leftCount) return kthIndex(t[leftRoot].left, t[rightRoot].left, tl, tm, k);
        return kthIndex(t[leftRoot].right, t[rightRoot].right, tm, tr, k - leftCount);
    }
    int kth(int l, int r, int k) const { return vals[kthIndex(roots[l], roots[r], 0, U, k)]; }
};
```

12. Problem-specific Segment Tree nodes already in this template

Existing card	Adaptation to the CP recursive pattern
3901 GCD Segment Tree	Scalar node; `merge = gcd(left,right)`, identity `0`, point update as base. Keep the problem's DP/state logic outside the tree.
3165 Maximum Sum of Non-adjacent Elements	Node has four endpoint states `s[leftSelected][rightSelected]`; merging two nodes maximizes over their inner boundary states while rejecting `left.rightSelected && right.leftSelected`. Point-update recursion stays the base.
3920 Maximize Fixed Points After Deletions	Tree stores a maximum DP value over transformed coordinates; merge is `max`, identity `0`, update is point-`chmax`, query is range max. Batch equal values before applying updates so equal-value states cannot chain.

Memory rule: first memorize the three recursive lines of thought: (1) disjoint / full-cover / partial-overlap, (2) recurse left and right at `tm`, (3) `pull` from children. Most variants change the node plus `merge`; lazy variants additionally change `apply`, `push`, and tag composition. See the CP-Algorithms Segment Tree reference for the source convention.

How the DSA sheet's Segment Tree questions map to these codes

Sheet problem	Best first template
Range Minimum Query	Generic Segment Tree or Sparse Table
Range Sum Query - Mutable	Fenwick or Generic Segment Tree
Create Sorted Array through Instructions	Fenwick on compressed values
Count of Range Sum	Fenwick + compression, or merge-sort-based counting
Count of Smaller Numbers After Self	Fenwick + compression
Advanced range update/query variants	Lazy Segment Tree

So you **do not need to blind-type six separate Segment Tree implementations** for those problems.

Reading-only: two short fallbacks

Bellman-Ford

```
// Guard INF before relaxation; an update on the nth pass means a reachable negative cycle exists.
```

```cpp
pair<bool,vector<long long>> bellman_ford(int s,int n,const vector<array<long long,3>>& e){
    const long long INF=4e18;vector<long long>d(n,INF);d[s]=0;bool neg=false;
    for(int it=0;it<n;++it){bool any=false;for(auto
     ↪ [u,v,w]:e)if(d[u]<INF&&d[v]>d[u]+w)d[v]=d[u]+w,any=true;if(!any)break;if(it==n-1)neg=true;}
    return {neg,d};
}
```

Floyd-Warshall

```cpp
// Keep k outermost so every update uses only paths whose intermediate vertices have already been
 ↪ processed.
void floyd(vector<vector<long long>>& d){
    int n=d.size();const long long INF=4e18;
    for(int k=0;k<n;++k)for(int i=0;i<n;++i)if(d[i][k]<INF)
        for(int j=0;j<n;++j)if(d[k][j]<INF)d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
}
```

What to move out of normal meal reading

These are valuable, but they should **not** consume your daily 10-20 minute reading window unless the OA signals them: - Mo's algorithm - Digit DP - Meet in the Middle - Suffix Array - Suffix Automaton - Aho-Corasick - Gaussian Elimination - SOS DP / FWHT - Li Chao / advanced D&C DP - Persistent Segment Tree - Merge Sort Tree - Segment Tree Beats - Full HLD implementation - Closest Pair - Eulerian Path / Hierholzer

These are now in this Reading Template's Advanced Code Vault. The goal is to know the trigger and reconstruct the idea, not to memorize every implementation.

Meal-reading rotation

Day	Meal reading
Mon	Arrays + binary search
Tue	Stack / hashing / strings
Wed	Graph shortest paths + topo
Thu	DSU / MST / trees
Fri	DP
Sat	Segment Tree + Fenwick variants
Sun	P1/P2 from Blind Typing or one weak area

Final placement rule

You should be able to look at a problem and answer these three questions before coding:

1. **What pattern is this?**
2. **What is the invariant/state stored by my template?**
3. **What single line / operation changes for the variant?**

That is a better use of your limited reading time than memorizing many nearly-duplicate implementations.

LeetCode Hard Stress-Test Track

These are the 22 Hards you supplied. They are **not** a second daily syllabus. For meal reading, take **one Hard at a time** and extract only: **invariant/state -> why the obvious approach fails -> key data structure/transition -> complexity**. The selected typing Hards already live in `blind_typing_codes_upgraded.md`; do not duplicate their full code here.

Problem	Reading mode	What to remember
420 Strong Password Checker	Type monthly + read	Greedy accounting; long strings require deletion priority by run length modulo 3.

Problem	Reading mode	What to remember
2612 Minimum Reverse Operations	Type every 2-4 weeks + read	BFS; reachable destinations form a parity-constrained interval; ordered sets let each state be removed once.
2945 Find Maximum Non-decreasing Array Length	Type monthly + read	Prefix sums + lower_bound + DP over the earliest valid previous block.
3165 Maximum Sum of Subsequence With Non-adjacent Elements	**Type weekly/biweekly + read**	Four boundary-selection states turn non-adjacent DP into a point-update Segment Tree.
3245 Alternating Groups III	Read	Dynamic circular runs + Fenwick/order-statistics style updates.
3276 Select Cells in Grid With Maximum Score	Read / occasional type	Small-state bitmask DP; recognize when dimensions make `2^k` feasible.
329 Longest Increasing Path in a Matrix	**Type weekly/biweekly + read**	Grid DFS becomes a DAG by strict increase; memoize each cell once.
3569 Maximize Count of Distinct Primes After Split	Read	SPF + occurrence positions + lazy Segment Tree; specialized but a strong range-update example.
3957 Maximum Sum of M Non-Overlapping Subarrays II	Read	Prefix DP + monotonic deque + WQS/penalty idea; advanced optimization rather than typing priority.
3504 Longest Palindrome After Substring Concatenation II	Read	Two-string palindrome DP and boundary matching.
3486 Longest Special Path II	Read / monthly reconstruction	Tree DFS + root-to-node sliding window + prefix distances; bookkeeping is more useful to understand than to memorize.
3463 Check If Digits Are Equal in String After Operations II	Read	Combinatorics modulo 10 via mod 2/mod 5 and CRT.
3357 Minimize Maximum Adjacent Element Difference	Type monthly + read	Binary-search-on-answer with a careful feasibility predicate around `-1` runs.
3563 Lexicographically Smallest String After Adjacent Removals	Read	Interval DP / structured removals.
3509 Maximum Product of Subsequences With an Alternating Sum Equal to K	Read	State DP where the alternating-sum state controls which positions can contribute to the product.
3704 Count No-Zero Pairs That Sum to N	Read	Digit DP with carry and validity/alive flags.
3630 Partition Array for Maximum XOR and AND	Read	Small-`n` subset/bitmask DP; inspect constraints before rejecting exponential state space.
3743 Maximize Cyclic Partition Score	Read	Cyclic partitioning reduced to DP over segment/extrema choices.
3901 Good Subsequence Queries	Type every 2-4 weeks + read	Point update + range GCD Segment Tree; focus on how a new operation plugs into the generic tree.
3920 Maximize Fixed Points After Deletions	Type monthly + read	Group equal values, transform to `d=i-value`, then run a LIS-like DP with Segment Tree queries.
3962 Maximum Subarray Sum After at Most K Swaps	Read	Greedy choice plus order statistics/BIT-style maintenance.
4055 Count Shadow Pairs II	Read	Divide-and-conquer crossing counts + coordinate compression + Fenwick.

Hard-reading rule

Do **not** try to finish all 22. A realistic rotation is **2-4 Hards per week**, with the same Hard revisited until you can explain the state transition without opening the solution. Only 8 are promoted into active typing because they materially strengthen reusable placement templates; the rest stay as solution-reading/stress-test problems.

Solved-Hard Benchmark Shelf: 18-27 minute solves

These eight are different from the "read this because it is too specialized" Hards. You already solved them completely inside a limited time budget, so the reading goal is **retention and transfer**. Read only the mechanism and revisit the full solution when your own approach starts becoming slow or fragile.

Problem	What to retain	Reading frequency
3139 Minimum Cost to Equalize Array	target enumeration + greedy pairing; compare `cost2` with `2*cost1`; account for parity/target cases	Every 2-4 weeks
3213 Construct String with Minimum Cost	`dp[i]` + efficient detection of which dictionary words end at `i` using hashing/Aho-Corasick	Monthly
3256 Maximum Value Sum by Placing Three Rooks I	only three rows/columns matter at a time; prune to top candidates instead of full column enumeration	Monthly
3260 Find the Largest Palindrome Divisible by K	constructive digit DP over remainder; left half determines the mirrored right half	Monthly / when doing modular DP
3266 Final Array State II	min-heap simulation until values balance, then cycle arithmetic + fast power	Every 2-4 weeks
3303 First Almost Equal Substring	prefix/suffix matching on a candidate window; natural KMP/Z transfer	Every 2-4 weeks
3609 Minimum Moves to Reach Target in Grid	work backwards; each step has a forced predecessor under the coordinate relation	Every 2-4 weeks
4007 Widest Possible Fence	frequency map + complementary pairing; enumerate candidate heights instead of constructing planks	Monthly

How these differ from blind typing

Because you already solved these, use them as **proof-of-speed benchmarks**, not as another set of solutions to memorize. Every 3-4 weeks, choose one at random and give yourself the original time constraint; if the structure appears quickly, stop after the accepted solution and record only the one line/transition that was hardest to recall.

The strongest transfer candidates for the Blind Typing file are **3139, 3266, 3303, and 3609**: they reinforce greedy case analysis, heap/cycle arithmetic, KMP/Z matching, and reverse-state reasoning respectively. The other four remain primarily reading/benchmark material because their exact constructions are less reusable. LeetCode confirms 3139 is a greedy/enumeration problem and 3213 uses DP with Aho-Corasick or hashing; 3266 combines heap simulation with periodicity and fast exponentiation; 3303 is explicitly a KMP/string-matching problem.

Sunday C++ / Quant / Developer companion

The Blind Typing file contains the detailed Sunday systems lab for active typing. When using this Reading Template, do **not** read every line. Instead, inspect the interfaces and lifetime rules: constructors/destructors, `virtual`/`override`, `unique_ptr`, raw observer pointers, templates, move semantics, const-correctness, and data-structure ownership.

The C++ lab is intentionally quant-flavored but should be understood as interview practice rather than a production trading-engine prescription. The quant-oriented goal is to remove fear of pointers/references, object lifetime, custom ordering, STL containers, and low-level memory concepts.

Advanced Code Vault - Moved from Blind Typing

These implementations were deliberately removed from Blind Typing. Use this section only when a placement/OA, target role, or a failed problem makes the technique relevant. The main meal-reading path remains the earlier compact sections.

3. Mo's Algorithm (Sqrt Decomposition)

Static range-query technique; remember the moving-window state machine before the ordering heuristic.

```
const int SQRT = 500;
struct MoQuery {
    int l, r, id;
    bool operator<(const MoQuery& o) const {
        return l / SQRT == o.l / SQRT ? ((l / SQRT) & 1 ? r < o.r : r > o.r) : l < o.l;
    }
};
```

3. Manacher's Algorithm

Linear longest-palindromic-substring template; useful when center expansion must be amortized.

```
vector<int> manacher(string s) {
    int n = s.size(), l = 1, r = 1; s = "$" + s + "^"; vector<int> p(n + 2);
    for (int i = 1; i <= n; i++) {
        p[i] = max(0, min(r - i, p[l + (r - i)]));
        while (s[i - p[i]] == s[i + p[i]]) p[i]++;
        if (i + p[i] > r) l = i - p[i], r = i + p[i];
    }
    return vector<int>(p.begin() + 1, p.end() - 1);
}
```

4. Lyndon Factorization (Min Cyclic Shift)

Use for minimum cyclic rotation / Lyndon decomposition; specialized but compact once understood.

```
string min_cyclic_string(string s) {
    s += s; int n = s.size(), i = 0, ans = 0;
    while (i < n / 2) {
        ans = i; int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) { s[k] < s[j] ? k = i : k++; j++; }
        while (i <= k) i += j - k;
    }
    return s.substr(ans, n / 2);
}
```

5. Combinatorics (nCr, Catalan, Stars & Bars) & Mint Struct

Reusable modular-combinatorics utilities; read the assumptions on MOD before using inverse by Fermat.

```
const int MOD = 1e9 + 7;
struct Mint {
    int v;
    Mint(long long x = 0) : v((x % MOD + MOD) % MOD) {}
    Mint& operator+=(Mint o) { if ((v += o.v) >= MOD) v -= MOD; return *this; }
    Mint& operator-=(Mint o) { if ((v -= o.v) < 0) v += MOD; return *this; }
    Mint& operator*=(Mint o) { v = (long long)v * o.v % MOD; return *this; }
    friend Mint operator+(Mint a, Mint b) { return a += b; }
    friend Mint operator-(Mint a, Mint b) { return a -= b; }
    friend Mint operator*(Mint a, Mint b) { return a *= b; }
    Mint pow(long long k) const {
        Mint res = 1, a = *this;
        while (k) { if (k & 1) res *= a; a *= a; k >>= 1; }
        return res;
    }
    Mint inv() const { return pow(MOD - 2); }
    Mint& operator/=(Mint o) { return *this *= o.inv(); }
    friend Mint operator/(Mint a, Mint b) { return a /= b; }
};

vector<Mint> fact, invf;
void precompute(int n) {
    fact.assign(n + 1, 1); invf.assign(n + 1, 1);
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * Mint(i);
    invf[n] = fact[n].inv();
```

```
    for (int i = n - 1; i > 0; i--) invf[i] = invf[i + 1] * Mint(i + 1);
}

Mint nCk(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * invf[k] * invf[n - k];
}
Mint catalan(int n) { return nCk(2 * n, n) * Mint(n + 1).inv(); }
Mint stars_bars(int n, int k) { return nCk(n + k - 1, k - 1); }
```

6. Matrix Exponentiation

Use for linear recurrences and fixed-length path counting; adapt the matrix size and multiplication operation.

```
template<size_t N>
struct Matrix {
    long long m[N][N] = {0};
    void identity() { for (size_t i = 0; i < N; ++i) m[i][i] = 1; }
    Matrix operator*(const Matrix& b) const {
        Matrix res;
        for (size_t i = 0; i < N; ++i) for (size_t k = 0; k < N; ++k) for (size_t j = 0; j < N; ++j)
            res.m[i][j] = (res.m[i][j] + m[i][k] * b.m[k][j]) % MOD;
        return res;
    }
    Matrix pow(long long p) const {
        Matrix res, base = *this; res.identity();
        while (p > 0) { if (p & 1) res = res * base; base = base * base; p >>= 1; }
        return res;
    }
};
```

3. Tarjan's Bridges & Articulation Points

DFS low-link template; bridges use low[v] > tin[u], articulation uses low[v] >= tin[u].

```
int timer = 0;
vector<int> tin, low; vector<bool> is_art; vector<array<int,2>> bridges;

void dfs_bridges(int u, int p, const vector<vector<int>>& adj) {
    tin[u] = low[u] = timer++; int child = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (tin[v] != -1) {
            low[u] = min(low[u], tin[v]);
        } else {
            dfs_bridges(v, u, adj);
            low[u] = min(low[u], low[v]);
            child++;
            if (low[v] > tin[u]) bridges.push_back({u, v});
            if (low[v] >= tin[u] && p != -1) is_art[u] = 1;
        }
    }
    if (p == -1 && child > 1) is_art[u] = 1;
}
```

4. Euler Tour Technique (Tree Flattening)

Turns subtree queries into interval queries; combine with Fenwick/Segment Tree.

```
int timer = 0;
vector<int> tin, tout;
void euler_tour(int u, int p, const vector<vector<int>>& adj) {
    tin[u] = ++timer;
    for (int v : adj[u]) {
        if (v != p) euler_tour(v, u, adj);
```

```
    }
    tout[u] = timer;
}
```

2. Bitmask DP Scaffold (Assignments / TSP)

Use when n is small enough for 2^n states; the assignment index is usually popcount(mask).

```
void bitmask_dp(int n, vector<vector<int>>& cost) {
    vector<int> dp(1 << n, 1e9);
    dp[0] = 0;
    for (int mask = 0; mask < (1 << n); ++mask) {
        if (dp[mask] == 1e9) continue;
        int bits_active = __builtin_popcount(mask);
        for (int i = 0; i < n; ++i) {
            if (!(mask & (1 << i))) {
                int next_mask = mask | (1 << i);
                dp[next_mask] = min(dp[next_mask], dp[mask] + cost[bits_active][i]);
            }
        }
    }
}
```

1. Suffix Array

Doubling construction; read when suffix ordering is required rather than using hashing/KMP.

```
vector<int> suffix_array(string s) {
    s += "$"; int n = s.size(); vector<int> p(n), c(n); vector<pair<char,int>> a(n);
    for (int i = 0; i < n; i++) a[i] = {s[i], i}; sort(a.begin(), a.end());
    for (int i = 0; i < n; i++) p[i] = a[i].second;
    for (int i = 1; i < n; i++) c[p[i]] = a[i].first == a[i - 1].first ? c[p[i - 1]] : c[p[i - 1]] + 1;

    for (int k = 0; (1 << k) < n; k++) {
        for (int i = 0; i < n; i++) p[i] = (p[i] - (1 << k) + n) % n;
        vector<int> np(n), nc(n), cnt(n, 0), pos(n);
        for (int x : p) cnt[c[x]]++;
        for (int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
        for (int x : p) np[pos[c[x]]++] = x;
        p = np; nc[p[0]] = 0;
        for (int i = 1; i < n; i++) {
            pair<int,int> cur = {c[p[i]], c[(p[i] + (1 << k)) % n]};
            pair<int,int> pre = {c[p[i - 1]], c[(p[i - 1] + (1 << k)) % n]};
            nc[p[i]] = cur == pre ? nc[p[i - 1]] : nc[p[i - 1]] + 1;
        }
        c = nc;
    }
    return vector<int>(p.begin() + 1, p.end());
}
```

2. Heavy-Light Decomposition (HLD)

Maps tree paths to O(log n) chain intervals; normally paired with a Segment Tree.

```
struct HLD {
    int n, timer; vector<int> sz, top, dep, par, in, out; vector<vector<int>> adj;
    HLD(int n) : n(n), timer(0), sz(n, 1), top(n), dep(n), par(n), in(n), out(n), adj(n) {}
    void dfs_sz(int u = 0, int p = -1, int d = 0) {
        par[u] = p; dep[u] = d; sz[u] = 1;
        if (adj[u].size() && adj[u][0] == p) swap(adj[u][0], adj[u].back());
        for (int& v : adj[u]) if (v != p) {
            dfs_sz(v, u, d + 1); sz[u] += sz[v];
            if (sz[v] > sz[adj[u][0]]) swap(v, adj[u][0]);
        }
    }
```

```
    void dfs_hld(int u = 0, int p = -1, int t = 0) {
        in[u] = timer++; top[u] = t;
        for (int v : adj[u]) if (v != p) dfs_hld(v, u, v == adj[u][0] ? t : v);
        out[u] = timer;
    }
    int lca(int u, int v) {
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = par[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }
};
```

3. Meet in the Middle

Split n into halves and enumerate 2^(n/2) states; useful for n around 40-46.

```
long long meet_in_the_middle(const vector<long long>& arr, long long target) {
    int n = arr.size(), m1 = n / 2, m2 = n - m1;
    auto get_sums = [&](int st, int len) {
        vector<long long> sums;
        for (int i = 0; i < (1 << len); i++) {
            long long cur = 0;
            for (int j = 0; j < len; j++) if (i & (1 << j)) cur += arr[st + j];
            sums.push_back(cur);
        }
        sort(sums.begin(), sums.end()); return sums;
    };
    auto sum1 = get_sums(0, m1), sum2 = get_sums(m1, m2);
    long long ans = 0;
    for (long long s1 : sum1)
        ans += upper_bound(sum2.begin(), sum2.end(), target - s1) -
               lower_bound(sum2.begin(), sum2.end(), target - s1);
    return ans;
}
```

1. BigInt (Rule of Five & Memory Leaks Test)

C++ lifetime drill: destructor, deep copy, move constructor, and move assignment.

```
class BigInt {
    int* data; size_t sz, cap;
public:
    BigInt(size_t c = 16) : sz(0), cap(c), data(new int[c]()) {}
    ~BigInt() { delete[] data; }

    BigInt(const BigInt& o) : sz(o.sz), cap(o.cap) {
        data = new int[cap]();
        for (size_t i = 0; i < sz; ++i) data[i] = o.data[i];
    }
    BigInt& operator=(const BigInt& o) {
        if (this != &o) {
            delete[] data;
            sz = o.sz; cap = o.cap;
            data = new int[cap]();
            for (size_t i = 0; i < sz; ++i) data[i] = o.data[i];
        }
        return *this;
    }

    BigInt(BigInt&& o) noexcept : data(o.data), sz(o.sz), cap(o.cap) {
        o.data = nullptr; o.sz = o.cap = 0;
    }
```

```
    BigInt& operator=(BigInt&& o) noexcept {
        if (this != &o) {
            delete[] data; data = o.data; sz = o.sz; cap = o.cap;
            o.data = nullptr; o.sz = o.cap = 0;
        }
        return *this;
    }
};
```

2. CRTP & Cache Line Alignment (Low Latency)

Static polymorphism and alignment vocabulary for low-latency C++ interviews.

```
// Static Polymorphism (Zero VTable overhead)
template <typename Derived>
struct MatchingEngine {
    void process() {
        static_cast<Derived*>(this)->execute();
    }
};

struct CryptoEngine : public MatchingEngine<CryptoEngine> {
    void execute() { /* Fast path */ }
};

struct alignas(64) OrderBookCounter {
    uint64_t fills;
};
```

Lazy Segment Tree: range assign + range sum

Assignment overrides an existing lazy add/set state; read tag precedence carefully.

```
// A pending assignment must be pushed before a pending add; assignment replaces the whole segment value.
struct AssignSum {
    int n; vector<long long> st,add; vector<long long> setv; vector<char> hasSet;
    AssignSum(int n):n(n),st(4*n),add(4*n),setv(4*n),hasSet(4*n){}
    void applySet(int v,int l,int r,long long x){st[v]=x*(r-l+1);setv[v]=x;hasSet[v]=1;add[v]=0;}
    void applyAdd(int v,int l,int r,long long x){st[v]+=x*(r-l+1);if(hasSet[v])setv[v]+=x;else
     ↪ add[v]+=x;}
    void push(int v,int l,int r){
        if(l==r){hasSet[v]=0;add[v]=0;return;}
        int m=(l+r)/2;
        if(hasSet[v]){applySet(v*2,l,m,setv[v]);applySet(v*2+1,m+1,r,setv[v]);hasSet[v]=0;}
        if(add[v]){applyAdd(v*2,l,m,add[v]);applyAdd(v*2+1,m+1,r,add[v]);add[v]=0;}
    }
    void update(int ql,int qr,long long x,bool isSet,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return;
        if(ql<=l&&r<=qr){if(isSet)applySet(v,l,r,x);else applyAdd(v,l,r,x);return;}
        push(v,l,r);int
 ↪ m=(l+r)/2;update(ql,qr,x,isSet,v*2,l,m);update(ql,qr,x,isSet,v*2+1,m+1,r);st[v]=st[v*2]+st[v*2+1];
    }
    long long query(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return 0;if(ql<=l&&r<=qr)return st[v];push(v,l,r);int m=(l+r)/2;return
         ↪ query(ql,qr,v*2,l,m)+query(ql,qr,v*2+1,m+1,r);
    }
};
```

Lazy Segment Tree: range affine transform + sum

Affine tags compose in chronological order; this is the canonical non-commutative lazy-tag example.

```
// For sum, (a*x+b) changes a segment sum to a*sum+b*len; compose new(a,b) after old(c,d) as (a*c,a*d+b).
struct AffineSum {
    struct Tag{long long a=1,b=0;}; struct Node{long long sum=0;int len=0;};
```

```
    int n;vector<Node>st;vector<Tag>lz;vector<char>has;
    AffineSum(const vector<long long>&a):n(a.size()),st(4*n),lz(4*n),has(4*n){build(1,0,n-1,a);}
    void build(int v,int l,int r,const vector<long long>&a){if(l==r){st[v]={a[l],1};return;}int
     ↪ m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);pull(v);}
    void pull(int v){st[v].sum=st[v*2].sum+st[v*2+1].sum;st[v].len=st[v*2].len+st[v*2+1].len;}
    void apply(int v,Tag
     ↪ t){st[v].sum=t.a*st[v].sum+t.b*st[v].len;lz[v]={t.a*lz[v].a,t.a*lz[v].b+t.b};has[v]=1;}
    void push(int v){if(!has[v])return;apply(v*2,lz[v]);apply(v*2+1,lz[v]);lz[v]={1,0};has[v]=0;}
    void update(int ql,int qr,Tag t,int v=1,int l=0,int
     ↪ r=-1){if(r<0)r=n-1;if(qr<l||r<ql)return;if(ql<=l&&r<=qr){apply(v,t);return;}push(v);int
     ↪ m=(l+r)/2;update(ql,qr,t,v*2,l,m);update(ql,qr,t,v*2+1,m+1,r);pull(v);}
    long long query(int ql,int qr,int v=1,int l=0,int r=-1){if(r<0)r=n-1;if(qr<l||r<ql)return
     ↪ 0;if(ql<=l&&r<=qr)return st[v].sum;push(v);int m=(l+r)/2;return
     ↪ query(ql,qr,v*2,l,m)+query(ql,qr,v*2+1,m+1,r);}
};
```

Segment Tree Beats: range chmin + range sum/max

Ordinary lazy propagation cannot represent arbitrary chmin on a segment; Beats tracks max/second-max/count.

```
// Store max, second max, count of max, and sum; chmin only changes the current maximum layer.
struct SegBeats {
    struct N{long long mx,sx,sum;int cnt;};
    int n; vector<N> st;
    SegBeats(const vector<long long>& a):n(a.size()),st(4*n){build(1,0,n-1,a);}
    N mergeN(N a,N b){
        N z;z.sum=a.sum+b.sum;
        if(a.mx==b.mx)z.mx=a.mx,z.cnt=a.cnt+b.cnt,z.sx=max(a.sx,b.sx);
        else if(a.mx>b.mx)z.mx=a.mx,z.cnt=a.cnt,z.sx=max(a.sx,b.mx);
        else z.mx=b.mx,z.cnt=b.cnt,z.sx=max(a.mx,b.sx);
        return z;
    }
    void build(int v,int l,int r,const vector<long long>& a){
        if(l==r){st[v]={a[l],LLONG_MIN,a[l],1};return;}int
         ↪ m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);st[v]=mergeN(st[v*2],st[v*2+1]);
    }
    void apply(int v,long long x){if(x>=st[v].mx)return;st[v].sum-=(st[v].mx-x)*st[v].cnt;st[v].mx=x;}
    void push(int v){apply(v*2,st[v].mx);apply(v*2+1,st[v].mx);}
    void chmin(int ql,int qr,long long x,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql||x>=st[v].mx)return;
        if(ql<=l&&r<=qr&&x>st[v].sx){apply(v,x);return;}
        if(l==r){apply(v,x);return;}
        push(v);int
 ↪ m=(l+r)/2;chmin(ql,qr,x,v*2,l,m);chmin(ql,qr,x,v*2+1,m+1,r);st[v]=mergeN(st[v*2],st[v*2+1]);
    }
    long long queryMax(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return LLONG_MIN;if(ql<=l&&r<=qr)return st[v].mx;push(v);int
         ↪ m=(l+r)/2;return max(queryMax(ql,qr,v*2,l,m),queryMax(ql,qr,v*2+1,m+1,r));
    }
    long long querySum(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return 0;if(ql<=l&&r<=qr)return st[v].sum;push(v);int m=(l+r)/2;return
         ↪ querySum(ql,qr,v*2,l,m)+querySum(ql,qr,v*2+1,m+1,r);
    }
};
```

Merge Sort Tree: count <= x in a range

Static range order-statistics structure; each covered node contributes a binary search.

```
// Each node stores a sorted vector; a query touches O(log n) nodes and binary-searches each one.
struct MergeSortTree {
    int n; vector<vector<int>> st;
    MergeSortTree(const vector<int>& a):n(a.size()),st(4*n){build(1,0,n-1,a);}
    void build(int v,int l,int r,const vector<int>& a){
```

```
        if(l==r){st[v]={a[l]};return;}int m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);merge(st[v*2].
        ↪ begin(),st[v*2].end(),st[v*2+1].begin(),st[v*2+1].end(),back_inserter(st[v]));
    }
    int leq(int ql,int qr,int x,int v=1,int l=0,int r=-1) const {
        if(r<0)r=n-1;if(qr<l||r<ql)return 0;
        if(ql<=l&&r<=qr)return upper_bound(st[v].begin(),st[v].end(),x)-st[v].begin();
        int m=(l+r)/2;return leq(ql,qr,x,v*2,l,m)+leq(ql,qr,x,v*2+1,m+1,r);
    }
};
```

Persistent Segment Tree: k-th number in subarray

Prefix roots represent versions; subtract root L-1 from root R to recover a subarray frequency tree.

```
// Root R minus root L-1 counts how many values fall in each value interval of the queried subarray.
struct PST {
    struct Node{int l=0,r=0,c=0;};
    vector<Node> tr; vector<int> roots,vals; int U;
    PST(int n,int U):tr(1),roots(n+1),U(U){}
    int clone(int x){tr.push_back(tr[x]);return tr.size()-1;}
    int add(int old,int l,int r,int p){
        int v=clone(old);++tr[v].c;if(l!=r){int m=(l+r)/2;if(p<=m)tr[v].l=add(tr[v].l,l,m,p);else
        ↪ tr[v].r=add(tr[v].r,m+1,r,p);}return v;
    }
    void build(const vector<int>& a){
        vals=a;sort(vals.begin(),vals.end());vals.erase(unique(vals.begin(),vals.end()),vals.end());U=
↪ vals.size();
        roots.assign(a.size()+1,0);for(int i=0;i<(int)a.size();++i)roots[i+1]=add(roots[i],0,U-1,
↪ lower_bound(vals.begin(),vals.end(),a[i])-vals.begin());
    }
    int kth(int A,int B,int l,int r,int k){
        if(l==r)return vals[l];int m=(l+r)/2,c=tr[tr[B].l].c-tr[tr[A].l].c;
        if(k<=c)return kth(tr[A].l,tr[B].l,l,m,k);return kth(tr[A].r,tr[B].r,m+1,r,k-c);
    }
};
```

1. Eulerian Path (Hierholzer's Algorithm)

Constructs an Euler trail by consuming edges and appending vertices on backtrack.

```
// Hierholzer's Algorithm (Directed Graph)
vector<int> find_eulerian_path(int n, vector<vector<int>>& adj) {
    vector<int> path, out(n), in(n);
    for (int u = 0; u < n; u++) { out[u] = adj[u].size(); for (int v : adj[u]) in[v]++; }
    int start = 0, end = 0, start_node = 0;
    for (int i = 0; i < n; i++) {
        if (out[i] - in[i] == 1) { start++; start_node = i; }
        else if (in[i] - out[i] == 1) end++;
        else if (in[i] != out[i]) return {};
    }
    if (!(start == 0 && end == 0) && !(start == 1 && end == 1)) return {};

    stack<int> st; st.push(start_node);
    while (!st.empty()) {
        int u = st.top();
        if (out[u] == 0) { path.push_back(u); st.pop(); }
        else { int v = adj[u][--out[u]]; st.push(v); }
    }
    reverse(path.begin(), path.end()); return path;
}
```

3. Closest Pair of Points (Divide & Conquer)

Geometry fallback; the sweep implementation keeps only points within the current distance band.

```cpp
struct pt { double x, y; };
double closest_pair(vector<pt>& p) {
    sort(p.begin(), p.end(), [](pt a, pt b) { return a.x < b.x; });
    set<pair<double, double>> s; double d = 1e9; int l = 0;
    for (int i = 0; i < p.size(); i++) {
        while (l < i && p[i].x - p[l].x >= d) { s.erase({p[l].y, p[l].x}); l++; }
        auto it_low = s.lower_bound({p[i].y - d, p[i].x - d});
        for (auto it = it_low; it != s.end() && it->first - p[i].y <= d; it++) {
            d = min(d, hypot(p[i].x - it->second, p[i].y - it->first));
        }
        s.insert({p[i].y, p[i].x});
    } return d;
}
```

5. Sum Over Subsets (SOS) DP & Variants

Subset/superset zeta transforms over 2^k states; learn the loop structure, then map it to the required transform.

```cpp
void sos_dp(int max_bits, vector<int>& a) {
    vector<int> dp(1 << max_bits, 0);
    for (int x : a) dp[x]++;

    for (int i = 0; i < max_bits; i++) {
        for (int mask = 0; mask < (1 << max_bits); mask++) {
            if (mask & (1 << i)) {
                dp[mask] += dp[mask ^ (1 << i)];
            }




        }
    }
}
```

6. Gaussian Elimination (Linear Systems & Expected Value)

General linear-system solver; inspect pivoting and floating-point tolerances before reuse.

```cpp
// Gaussian Elimination O(N^3)
// Returns: 1 (Unique solution), 0 (No solution), INF (Infinite solutions)
int gauss(vector<vector<double>>& a, vector<double>& ans) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i) if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
        if (abs(a[sel][col]) < 1e-9) continue;
        swap(a[sel], a[row]); where[col] = row;
        for (int i = 0; i < n; ++i) if (i != row) {
            double c = a[i][col] / a[row][col];
            for (int j = col; j <= m; ++j) a[i][j] -= a[row][j] * c;
        }
        ++row;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; ++i) if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i = 0; i < n; ++i) {
        double sum = 0; for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
        if (abs(sum - a[i][m]) > 1e-9) return 0;
    }
    for (int i = 0; i < m; ++i) if (where[i] == -1) return 1e9;
    return 1;
}
```

7. Advanced DP Optimizations (Li-Chao & D&C)

Li-Chao handles dynamic line minima; divide-and-conquer DP needs monotone optimal split points.

```
// Li-Chao Tree (Convex Hull Trick Alternative) O(log X)
struct LiChao {
    struct Line { ll m, c; ll operator()(ll x) { return m * x + c; } };
    struct Node { Line seg; Node *l, *r; Node(Line s): seg(s), l(0), r(0) {} };
    Node* root; int N;
    LiChao(int n): N(n) { root = new Node({0, (ll)1e18}); }
    void upd(Node*& n, int l, int r, Line y) {
        if (!n) { n = new Node(y); return; }
        int m = (l + r) / 2; bool b1 = y(l) < n->seg(l), b2 = y(m) < n->seg(m);
        if (b2) swap(n->seg, y);
        if (l + 1 == r) return;
        if (b1 != b2) upd(n->l, l, m, y); else upd(n->r, m, r, y);
    }
};

void dac_opt(int i, int l, int r, int ql, int qr, vector<vector<int>>& dp) {
    if (l > r) return;
    int mid = (l + r) / 2; array<int,2> bst = {1e9, -1};
    for (int k = ql; k <= min(mid, qr); k++)
        bst = min(bst, {dp[i - 1][k - 1] + cost(k, mid), k});
    dp[i][mid] = bst[0];
    dac_opt(i, l, mid - 1, ql, bst[1], dp);
    dac_opt(i, mid + 1, r, bst[1], qr, dp);
}
```

Digit DP - canonical count-by-digit-sum

```
// State = position, accumulated digit sum, and tight flag; memoize only non-tight states when useful.
long long count_digit_sum(const string& s, int target) {
    int n = s.size();
    long long memo[20][172];
    for (auto& row : memo) for (auto& x : row) x = -1;
    function<long long(int,int,bool)> dfs = [&](int pos, int sum, bool tight) -> long long {
        if (sum > target) return 0;
        if (pos == n) return sum == target;
        if (!tight && memo[pos][sum] != -1) return memo[pos][sum];
        int lim = tight ? s[pos] - '0' : 9;
        long long ans = 0;
        for (int d = 0; d <= lim; ++d) ans += dfs(pos + 1, sum + d, tight && d == lim);
        if (!tight) memo[pos][sum] = ans;
        return ans;
    };
    return dfs(0, 0, true);
}
```

Aho-Corasick - complete multi-pattern matcher

```
// Build a trie plus failure links; each text character advances in O(1) amortized time.
struct AhoCorasick {
    struct Node { array<int,26> next{}; int link=0, out=0; Node(){next.fill(-1);} };
    vector<Node> t{Node()};
    void add(const string& s){
        int v=0;
        for(char ch:s){int
         ↪  c=ch-'a';if(t[v].next[c]==-1){t[v].next[c]=t.size();t.push_back(Node());}v=t[v].next[c];}
        ++t[v].out;
    }
    void build(){
        queue<int> q;
        for(int c=0;c<26;++c){int u=t[0].next[c];if(u==-1)t[0].next[c]=0;else q.push(u);}
```

```
        while(!q.empty()){int v=q.front();q.pop();t[v].out+=t[t[v].link].out;for(int c=0;c<26;++c){int
         ↪ u=t[v].next[c];if(u==-1)t[v].next[c]=t[t[v].link].next[c];else
         ↪ t[u].link=t[t[v].link].next[c],q.push(u);}}
    }
    long long matches(const string& s){long long ans=0;int v=0;for(char
     ↪ ch:s)v=t[v].next[ch-'a'],ans+=t[v].out;return ans;}
};
```

Suffix Automaton - compact reference

```
// Each state represents an end-position equivalence class; suffix links connect to the largest proper
 ↪ suffix class.
struct SuffixAutomaton {
    struct State { int next[26]; int link=-1,len=0; State(){fill(next,next+26,-1);} };
    vector<State> st; int last=0;
    SuffixAutomaton(const string& s){st.reserve(2*s.size());st.push_back(State());for(char
 ↪ c:s)extend(c-'a');}
    void extend(int c){
        int cur=st.size();st.push_back(State());st[cur].len=st[last].len+1;int p=last;
        while(p!=-1&&st[p].next[c]==-1)st[p].next[c]=cur,p=st[p].link;
        if(p==-1)st[cur].link=0;else{int
         ↪ q=st[p].next[c];if(st[p].len+1==st[q].len)st[cur].link=q;else{int
         ↪ clone=st.size();st.push_back(st[q]);st[clone].len=st[p].len+1;while(p!=-1&&st[p].next[c]==q)
         ↪ st[p].next[c]=clone,p=st[p].link;st[q].link=st[cur].link=clone;}}
        last=cur;
    }
};
```